

Evaluating the performance of TinyML singular and ensemble techniques for intrusion detection in IoT networks

Abderahmane Hamdouchi^a, Ali Idri^{b,*}

^a Vanguard Center, Mohammed VI Polytechnic University, Benguerir, Morocco

^b Software Project Management Research Team, ENSIAS, Mohammed V University in Rabat, Morocco

ARTICLE INFO

Keywords:

Singular machine learning
Ensemble machine learning
TinyML
Internet of Things
Intrusion detection systems
Arduino UNO

ABSTRACT

As the Internet of Things (IoT) expands, safeguarding IoT networks from vulnerabilities becomes critical. Intrusion detection systems (IDS) leveraging machine learning (ML) techniques are essential for enhancing security and preventing unauthorized access. However, transmitting data to the cloud can introduce latency, impeding real-time attack detection. This research evaluates three TinyML ensemble techniques (random forest, XGBoost, and extra trees) and three singular techniques (decision tree, Gaussian naive Bayes, and multilayer perceptron) using two feature selection methods (maximum relevance minimum redundancy and analysis of variance) on the NF-ToN-IoT-v2 and NF-BoT-IoT-v2 datasets for cyberattack detection. Evaluations on the Arduino UNO used the prediction performance criteria (Cohen's kappa and Matthew's correlation coefficient), device metrics (latency, static RAM, and flash memory), and the Scott-Knott test and Borda count voting system to assess the statistical significance and to rank the models. Results show that singular TinyML models outperformed ensemble models for multiclass classification in the IDS-IoT context. The best models are: (1) MLP with 20 features and a hidden layer size of 56 for NF-ToN-IoT-v2; and (2) ET with 13 features, 2 estimators, and a tree depth of 16 for NF-BoT-IoT-v2.

1. Introduction

The Internet of Things (IoT) presents novel opportunities for a diverse array of implementations across sectors such as environmental monitoring, industry, smart residences, and agriculture [1]. Notwithstanding these advantages, an elevated susceptibility to cybersecurity breaches has present [2]. Implementing comprehensive security measures for IoT systems is challenging owing to their intricate and diverse characteristics. Approaches to address this obstacle have yielded methodologies that prioritize the safeguarding of data privacy, authentication of user identities, control of access within IoT networks, and preservation of confidence and confidentiality between users and devices [2]. Despite these endeavors, IoT networks continue to be susceptible to cyber threats. Hence, IoT-specific security tools that are specifically engineered to meet this demand are imperative. The integration of intrusion detection systems (IDS) into the IoT infrastructure may constitute a substantial stride in the direction of bolstering security [3].

IDS play a critical role in ensuring the security of the IoT, as highlighted in numerous computer science studies [3,4]. Machine learning

(ML) is pivotal for the development of IDS that can detect threats. Although ML-driven IDS technology demonstrates efficacy in traditional networks, it is inadequate in the IoT context. This shortcoming results from its failure to adjust to the complex and diverse IoT environment, which includes devices with limited resources and the need for immediate threat detection [5].

Conventional IDS-ML systems impose a constraint on the autonomy of IoT devices by requiring them to sustain an almost continuous connection to the cloud or central device. A novel methodology known as Tiny Machine Learning (TinyML) involves the direct installation of optimized ML models onto devices with restricted processing capabilities, such as those powered by microcontrollers. This process entails modifying ML models that were originally developed for more robust platforms to function on these diminutive devices [1,5]. Consequently, IoT devices acquire the capability to autonomously perform ML tasks, allowing the incorporation of ML models into embedded devices without requiring ongoing external connectivity [1,5].

The review [6] investigated seven studies conducted between 2021 and 2023 on utilizing TinyML for anomaly detection, with a specific emphasis on ML and deep learning (DL) on embedded devices. Sicari

* Corresponding author at: Mohammed V University, Rabat, Morocco.

E-mail addresses: abderahmane.hamdouchi@um6p.ma (A. Hamdouchi), Ali.Idri@um5.ac.ma (A. Idri).

<https://doi.org/10.1016/j.micpro.2025.105172>

Received 31 January 2025; Received in revised form 18 March 2025; Accepted 2 June 2025

Available online 3 June 2025

0141-9331/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

et al. [7] introduced a security-aware IoT platform based on a dual-layer fog computing architecture, while Attou et al. [8] proposed a cloud-based intrusion detection model using Random Forest (RF) and feature engineering. However, both studies [7,8] do not sufficiently address memory optimization and real-time attack detection using resource-constrained microcontrollers (MCU). Tekin et al. [9] analyzed the energy consumption of various ML techniques, including Decision Tree (DT), k-Nearest Neighbor (KNN), Artificial Neural Network (ANN), and Random Forest (RF), when used for intrusion detection, utilizing the DS2OS dataset and evaluating these models on different platforms such as Azure Cloud, a Dell laptop, and a Raspberry Pi, which is a small computing device with a central processing unit (CPU), without incorporating hyperparameter tuning during model training. Dehrouyeh et al. [10] conducted an experimental study on using TinyML to enhance cybersecurity in electric vehicle charging infrastructure (EVCI), utilizing the CICIDS2017 dataset and deploying a multilayer perceptron (MLP) model on the ESP32 microcontroller, comparing its performance with a RF model that was not deployed on an embedded device, without making comparisons between different TinyML models with various hyperparameter settings. However, these studies [6,9,10] have several limitations: (1) they predominantly employ default parameters for each ML algorithm, neglecting the examination of the potential impact of hyperparameters on the implementation of TinyML models in IDS scenarios; (2) there is no comparison of models in multiclassification scenarios with respect to accuracy, speed, and memory consumption; (3) the effects of feature thresholds and the hyperparameters of the TinyML techniques have not been investigated; (4) no research has utilized the Arduino UNO microcontroller, known for its minimal memory usage and low cost, to deploy models with various configurations for constructing an optimal IDS with minimal resources, aiming to generalize the models for any embedded device; and (5) no studies have explored the generalization of TinyML-based IDS across multiple datasets to assess their adaptability and performance in different contexts. To the best of our knowledge, there is currently no comprehensive research that investigates the effects of various feature thresholds and hyperparameters of TinyML models on memory consumption, response time, and cyber-attack detection, specifically for low-cost devices, such as the Arduino UNO and cross. This research gap prompted us to conduct the present study.

This study aims to address existing research gaps by examining and comparing the impact of two FS techniques on multiclass classification performance: maximum relevance minimum redundancy (mRMR) [11] for categorical data, and analysis of variance (ANOVA) [12] for numerical data. These FS methods are assessed using four thresholds (20 %, 40 %, 60 %, and 100 %) with three singular ML algorithms: decision tree (DT) [13], Gaussian naive Bayes (NB) [14], and multilayer perceptron (MLP) [15], as well as three ensemble ML techniques: random forest (RF) [16], XGBoost (XGB) [17], and extra trees (ET) [18], each with various hyperparameters. The empirical evaluations were over the two new NetFlow IoT datasets, NF-ToN-IoT-v2 and NF-BoT-IoT-v2 [19], using a highly constrained device (Arduino UNO) with minimal memory, to generalize the findings to other IoT embedded devices.

The selection of these FS techniques is based on their low computational cost, effectiveness in feature subset selection, and widespread application across various domains [20–22]. Given the heterogeneous nature of IoT data, this study employs a hybrid approach by combining ANOVA for numerical features and mRMR for categorical features. ANOVA evaluates the statistical significance of numerical features in relation to categorical targets, ensuring the selection of highly discriminative features. mRMR, on the other hand, balances feature relevance and redundancy, making it particularly effective for correlated categorical features [22]. The six classifiers are chosen based on their demonstrated effectiveness in attack detection [4,21] and suitability for deployment on embedded devices. The feature thresholds are selected based on their success in previous studies [21,23–25].

In this research, we assess 288 models for multiclass classification,

comprising 48 models for each ML technique ($288 = 6 \text{ ML techniques} \times 48 = 2 \text{ datasets} \times 4 \text{ feature thresholds} \times 6 \text{ hyperparameters per ML technique}$). We utilize a 5-fold validation approach for testing. Model performance is evaluated using two performance metrics: Cohen's kappa (Kappa) [26] and Matthew's correlation coefficient (MCC) [27], along with measuring prediction speed (latency) [28], memory usage (flash memory (FM) [29], and static random-access memory (SRAM) [30]) in the deployment environment (i.e. Arduino UNO device). The Scott–Knott (SK) test [31] is used to cluster the classifiers and determine the most stable cluster through a statistical comparison of their performances. Additionally, the Borda count (BC) [32] ranking system identifies the highest-performing models based on the five the performance criteria (MCC, Kappa, latency, SRAM, and FM).

The present study addresses the following research questions (RQs):

- **(RQ1):** What is the impact of the hyperparameters on the efficacy of the TinyML singular models implemented on the Arduino UNO device?
- **(RQ2):** What is the impact of the hyperparameters on the efficacy of the TinyML ensemble models implemented on the Arduino UNO device?
- **(RQ3):** Which TinyML technique, model parameters, and feature threshold are optimal for constructing the most effective IDS on the Arduino UNO device?

The primary contributions of this study are outlined as follows:

1. Analyzing the effectiveness of various ML techniques for intrusion detection in IoT environments.
2. Implementing TinyML models with varying parameters on the Arduino UNO platform, a highly resource-constrained embedded device, helps generalize the results and experimentation to IoT devices with greater resources than the Arduino UNO.
3. Investigating the impact of hyperparameters on the performance of TinyML models across various algorithms.
4. Assessing the effectiveness of FS techniques and ML algorithms in IDS within the realms of IoT and TinyML.
5. Identifying the most suitable TinyML-based approach for intrusion detection in IDS in IoT networks.

The remainder of this paper is organized as follows: [Section 2](#) provides an overview of the TinyML technology, along with the FS and ML techniques used [Section 3](#) reviews prior research on the utilization of TinyML in IDS for IoT. [Section 4](#) outlines the datasets used, performance metrics employed, SK test, BC voting system, and the research methodology applied in this investigation. [Section 5](#) discusses the findings. [Section 6](#) addresses the limitations and validity of this study. Finally, [Section 7](#) concludes the paper and proposes directions for future research.

2. Background

This section provides an overview of TinyML technology, FS techniques, and ML techniques employed in this study.

2.1. TinyML

TinyML consists of ML techniques optimized for use in low-power tiny devices such as microcontrollers. This enables the implementation of ML models on embedded systems with constraints on power, memory, and processing capabilities. TinyML performs duties requiring efficient local data processing, conserving energy, and eliminating the need for constant connectivity. Sophisticated hardware and software are utilized in this domain to facilitate effective on-device analytics for various purposes, including sensor data processing, vision, and audio [33].

2.2. Feature selection

In our research, we reduced the number of dimensions using FS, a common technique that seeks to identify the minimal set of informative features possible. The three primary categories of FS techniques are filter, embedded, and wrapper techniques. We utilized two well-known FS methods in our research: mRMR for categorical features, and ANOVA for numerical features.

a) Maximum relevance minimum redundancy (mRMR): is a filter-based selection method that selects highly predictive and uncorrelated features [11]. Relevance is calculated using the F-statistic for continuous features and mutual information for discrete features. Redundancy is measured using the Pearson correlation coefficient for continuous features and mutual information for discrete features. In this study, mRMR was used for categorical features.

b) Analysis of variance (ANOVA) is a statistical hypothesis test that uses parametric statistics to determine whether the means of two or more data samples come from the same distribution. ANOVA is used when one variable is numeric, and the other is categorical [12].

2.3. Machine learning techniques

This section provides an overview of the classification techniques used in our experiments. We employed four single techniques, DT, NB, and MLP, along with three ensemble techniques, RF, ET, and XGB.

a) Decision Tree (DT): is a hierarchical structure used to classify instances based on their attribute values. Each node in a DT represents a feature in an instance awaiting classification. Each branch represents a distinct decision rule, and each leaf node represents the final classification result. The root node, located at the top of the tree, symbolizes the feature that most efficiently partitions the training data [13]. A critical parameter to consider when implementing a DT is the maximum depth of the tree.

b) Gaussian Naive Bayes (NB): is a probability-based ML classification technique. It assumes that each class in the dataset follows a normal distribution and that every parameter independently contributes to the prediction of the output variable. This method estimates the probability that the dependent variable belongs to each class. A critical parameter to consider in this technique is the smoothing variable.

c) Multilayer Perceptron (MLP): is a feedforward artificial neural network (ANN) designed for classification and regression tasks. This architecture is widely regarded as the foundation for deep neural networks (DNN) and DL [15]. Two critical parameters to consider when implementing an MLP [34] are: (1) the number of hidden layers and (2) the number of neurons in the hidden layers.

d) Random Forest (RF): is a widely recognized ensemble learning algorithm consisting of numerous DT, each trained on randomly chosen subsets of data [16]. When employing the RF model, the following hyperparameters must be considered [35]: (1) the number of trees and (2) the maximum depth of each DT.

e) XGBoost (XGB): Short for eXtreme Gradient Boosting, XGBoost is a popular and robust open-source library used to implement gradient boosting decision trees. The core concept of gradient boosting involves training a sequence of weak models, each designed to correct the errors of its predecessor [17]. XGB offers several notable features that enhance its effectiveness compared with other gradient-boosting libraries [35]. The two main hyperparameters for XGB are: (1) the number of estimators and (2) the maximum depth of the tree.

f) Extra Trees (ET): Short for Extremely Randomized Trees, ET is an ensemble machine learning method [18]. It generates numerous unpruned decision trees from training data. For regression tasks, predictions are made by averaging the outputs of these trees, whereas majority voting is used for classification tasks. Unlike the

bagging and random forest (RF) algorithms, which construct each decision tree (DT) from a bootstrap sample of the training data, ET fits each decision tree using the entire training dataset. The two main hyperparameters for ET are: (1) the number of estimators and (2) the maximum depth of the tree.

3. Related work

Numerous studies have explored anomaly detection across different applications, notably in the IoT domain, by using various TinyML approaches. This section presents an overview of key studies focusing on the utilization of TinyML techniques for intrusion detection in IoT.

Numerous studies have focused on intrusion detection using TinyML. Tekin et al. [9] compared on-device ML algorithms with a focus on energy consumption. Their analysis assessed the training phase across IoT devices, cloud computing, and edge computing. During the inference phase, Raspberry Pi was used to evaluate the performance of various TinyML models, including Logistic Regression (LR), NB, KNN, DT, RF, and ANN, on the DS2OS dataset. The training environment prioritized accuracy, whereas both the inference and training environments evaluated the time and energy consumption. The DT algorithm emerged as the most effective model, achieving 99 % accuracy, with training and inference times of 17.46 microseconds (μ s) and 0.34 μ s, respectively, and training and inference energy consumptions of 178.73 Joules (J) and 4.29 J, respectively. Although this study significantly contributes to the field of TinyML for IDS, it has some limitations. For example, using Raspberry Pi as an end device, which functions as a small computer with a central processing unit (CPU), may not be the best option for TinyML applications. Instead, microcontroller units (MCUs) such as the Arduino UNO, which have limited resources, could be more suitable. In addition, using the default parameters provided by the Sklearn library in a TinyML environment may not consistently result in optimal performance. For instance, Sanchez et al. [1] demonstrated that the high memory consumption (SRAM) of an Arduino UNO rendered a support vector machine (SVM) with a radial basis function (RBF) kernel unusable; however, implementing a linear kernel was feasible. Hussain et al. [36] developed an IoT TinyML approach using a trained Convolutional Neural Network (CNN) model deployed on a Raspberry Pi. TensorFlow was used to build the model and TensorFlow Lite (TFLite) was used to implement it on an IoT device. This model aimed to detect two types of jamming attacks and normal behavior. Equipped with software-defined radio (SDR), the Raspberry Pi continuously monitored the Wi-Fi channel and gathered the Received Signal Strength (RSS) readings, which the TinyML model analyzed for jamming detection. Using privately collected data, the system achieved an accuracy of 86.33 % during testing, thereby highlighting the effectiveness of the TinyML-based detection method. However, the study had some limitations, such as using Raspberry Pi as an end device, which may not accurately represent low-resource environments due to its CPU capabilities. Additionally, the study relied solely on accuracy as a performance metric without considering other relevant measures for embedded devices. Dehrouyeh et al. [10] conducted an experimental case study focusing on enhancing cybersecurity in EVCI using TinyML. They converted an MLP model into TinyML format using TFLite and compared its performance with that of RF. The comparison was based on accuracy using a 5-fold cross-validation strategy, inference time, memory usage during inference, and model size. The evaluation used the CICIDS2017 dataset, and the Tiny_MLP model was deployed on an ESP32 microcontroller. The Tiny_MLP setup achieved an accuracy of 80.83 %, inference time of 246.825 μ s, 0.9023 kilobytes of memory during inference, and model size of 8.6953 kB. This study made a significant contribution to the TinyML IDS field. However, it is important to note that the study explored only one configuration of TinyML (Tiny_MLP) generated by the TFLite framework. The study did not include the Tiny_RF model because it was not converted or deployed for comparison, and it was limited to reducing the size of the RF model by reducing the number of estimators

from 100 to 10. Other conversion methods, such as MicroMLGen and Emlearn, were not explored in this study.

Al-Waisi et al. [37] conducted a study comparing various ML methodologies, including cloud-based, edge-based, and device-based strategies, for both training and detection purposes. They trained ML techniques such as K-nearest neighbor (K-NN), decision tree (DT), random forest (RF), and naive Bayes (NB) on two private datasets to detect resource-constrained attacks. Training and detection were performed in four different environments: Raspberry Pi, Arduino Nano, edge device, and cloud computing. The ML models were converted using TFLite. The findings indicated that the DT algorithm deployed on smart devices outperformed other approaches in terms of training efficiency, resource utilization, and the ability to detect resource-constrained attacks on IoT devices. The DT algorithm achieved an accuracy of 96.9 % across all ML methodologies evaluated for detecting resource-constrained attacks in IoT systems. Katib et al. [38] proposed the DL-TinyML-Driven Real-time Anomaly Detection for Predictive Maintenance (DLTML-RTADPM) technique to enhance the security of IoT consumer devices. The methodology included FS using the Fennec Fox Optimization Algorithm (FFA), anomaly detection with the Gradient Least Mean Squares-Bidirectional Long Short-Term Memory (GLMS-BiLSTM) model, and hyperparameter tuning via the Jaya Optimization Algorithm (JOA). The study evaluated the approach using the ToN-IoT dataset, where the DLTML-RTADPM model achieved a 98.11 % accuracy, surpassing models such as LSTM, CNN-LSTM, and GRU-DAE. Additionally, it demonstrated improved computational efficiency, with a significantly lower computational time (CT) (10.60 s), making it suitable for real-time anomaly detection in IoT environments. While the model was converted for TinyML compatibility, it was not deployed on a specific embedded device, focusing instead on optimizing its performance for resource-constrained environments. Al-Waisi and Soderi [39] conducted a study on detecting resource-constrained attacks in IoT smart devices, specifically targeting energy and memory attacks. They compared TinyML-based real-time detection with traditional ML techniques using a dataset collected from real-world experiments on Raspberry Pi and Arduino under normal and attack conditions. RF and SVM were employed for attack detection, with RF achieving the highest accuracy. The TinyML-based approach outperformed traditional methods, achieving 99.4 % accuracy on Raspberry Pi and 99.2 % on Arduino, along with lower false alarm (1.23 %) and misdetection rates (1.60 %). The findings demonstrated the effectiveness of TinyML in improving attack detection on low-power, resource-constrained IoT devices.

Several studies have examined the implementation of standard IDS on cloud-based and edge devices. Sicari et al. [7] proposed a security-aware IoT platform leveraging fog computing principles to enhance data dissemination efficiency and security in IoT environments. The platform employs a publish and subscribe protocol based on MQTT, coupled with a network of brokers, to manage data sharing. The architecture includes two fog layers: the first layer consists of Networked Smart Objects (NOSs) that acquire and process data, while the second layer comprises brokers responsible for disseminating the processed data to end-users. The platform integrates security mechanisms such as sticky policies, Authenticated Publish and Subscribe System (AUPS), and key management systems to ensure secure data sharing. A test campaign was conducted using a prototypical implementation with four NOS instances running on Raspberry Pi devices and a variable number of brokers (from 2 to 10). The results showed that increasing the number of brokers reduced the CPU load on brokers and decreased network latency, demonstrating better load balancing and improved system efficiency. However, the CPU load on NOSs remained constant, as they processed the same amount of data regardless of the number of brokers. Hanaa Attou et al. [8] proposed a cloud-based intrusion detection model using a RF classifier and feature engineering to improve cloud security. The model was evaluated on the NSL-KDD and Bot-IoT datasets, achieving accuracy rates of 98.3 % and 99.99 %, respectively. By leveraging data visualization and preprocessing, the approach reduced

the number of features, enhancing efficiency and execution time. The RF classifier outperformed other machine learning models, including DNN, LSTM, and SVM, even though they use metrics sensitive to imbalanced data or a cost-sensitive strategy to address this issue, and despite working with highly imbalanced datasets. However, the recall metric on the NSL-KDD dataset remains a limitation, suggesting that future research could explore deep learning and ensemble methods for further improvement. These studies [7,8] provide valuable contributions in terms of performance; however, they lack sufficient optimization for memory efficiency and real-time execution. In contrast, our study focuses on implementing a TinyML model on resource-constrained devices, optimizing memory usage to ensure deployment across various IoT devices while enhancing cybersecurity protection.

Table 1 provides a summary of four selected studies that explored the application of TinyML for intrusion detection in IoT environments.

4. Experimental design

This section describes the datasets, performance metrics, BC system, and methodology used to analyze and compare the ensemble TinyML models.

4.1. Datasets description

Fig. 1 illustrates the process of generating traffic flow using *nProbe*, a tool created by *Ntop* that follows the NetFlow standard version 9. This tool extracts various attributes from network flow data in 'pcap' format and labels the generated CSV files. Researchers can utilize these attributes to evaluate or train ML models. In this study, two IoT-IDS datasets containing a shared set of features were used.

a) The NF-ToN-IoT-v2 dataset is created from the ToN-IoT dataset [40] by using publicly available 'pcaps' to generate NetFlow records tailored for IoT networks. It contains 16,940,496 data flows, with 63.99 % (10,841,027) identified as attack samples and 36.01 % (6,099,469) labeled as benign. Intrusion events, accounting for 36.01 %, are spread across various categories, resulting in an imbalanced distribution, as shown in Fig. 2. b. This dataset includes eight attack categories. Furthermore, Fig. 2.a. displays the distribution of all classes in NF-BoT-IoT-v2.

b) The NF-BoT-IoT-v2 dataset is an IoT NetFlow-based dataset derived from the original BoT-IoT dataset [41]. Feature extraction is performed on publicly available 'pcap' files, and each flow is labeled with its corresponding attack category. The dataset contains 37,763,497 data flows, with 99.64 % (37,628,460) classified as attack samples and 0.36 % (135,037) labeled as benign. The intrusion events make up 99.64 % of the dataset and are distributed with an imbalanced percentage, as shown in Fig. 2.c. It comprises four attack categories. In addition, Fig. 2. a. illustrates the distribution of all classes in the NF-BoT-IoT-v2.

As illustrated in Fig. 2, the NF-ToN-IoT-v2 and NF-BoT-IoT-v2 datasets exhibit class imbalance, a common challenge in real-world IoT scenarios. Our study specifically addresses this issue by employing performance metrics such as MCC and kappa, which are less affected by data imbalance [42]. These metrics ensure that the models remain effective in detecting both frequent and rare cyberattacks.

4.2. Performance criteria

Matthew's Correlation Coefficient (MCC) [27] and Cohen's kappa (Kappa) [26] are defined in Eqs. (1) and (2), respectively:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (1)$$

Table 1
Summary of the literature review.

Paper	Dataset	Algorithm	Device	Performance	Finding
Tekin et al. [9]	DS2OS	LR KNN DT RF NB ANN	Raspberry Pi	Accuracy = 99 % Training time = 17.46 μ s Inference time = 0.34 μ s Training energy = 178.73 J Inference energy = 4.29 J.	Performed a comparative study of TinyML focusing on energy usage for intrusion detection in IoT applications, utilizing the DS2OS dataset on the Raspberry Pi. The results showed that DT was the most efficient choice.
Hussain et al. [36]	Private dataset	CNN	Raspberry Pi	Accuracy = 86.33 %	Created a TinyML solution for IoT applications by implementing a CNN model on a Raspberry Pi, tailored for identifying jamming. Utilizing proprietary data, the system attained an accuracy of 86.33 % during testing.
Dehrouyeh et al. [10]	CICIDS2017	MLP RF	ESP32	Accuracy = 80.83 % Inference time = 246.825 μ s Inference memory = 0.9023 KB Model size = 8.6953 kb	Conducted a comparison of TinyML to improve cybersecurity in EVCI, utilizing the CICIDS2017 dataset on the ESP32 microcontroller. The findings indicated that the MLP proved to be the most effective option.
Al-Waisi et al. [37]	Private dataset	K-NN DT RF NB	Raspberry Pi Arduino Nano	Accuracy = 96.9 % Inference Time is 24.6 s for Arduino Nano and 27.5 s for Raspberry Pi	Performed a comparison of TinyML to identify resource-constrained attacks, using a private dataset on both the Raspberry Pi and Arduino Nano. The results showed that DT were the most effective choice.
Katib et al. [38]	ToN-IoT	DLTML-RTADPM	No-Device	Accuracy = 98.11 % Recall = 89.36 % Precision = 90.30 % F1-Score = 89.74 % CT = 10.60 s	Developed a TinyML-based anomaly detection model for IoT security, utilizing the ToN-IoT dataset. The DLTML-RTADPM model, incorporating GLMS-BiLSTM and FFA-based FS, achieved 98.11 % accuracy with optimized computational efficiency.
Al-Waisi and Soderi [39]	Private dataset	RF, SVM	Raspberry Pi, Arduino	Accuracy = 99.4 % Raspberry Pi, 99.2 % Arduino	Conducted a study on detecting resource-constrained attacks using TinyML and traditional ML techniques. TinyML-based models outperformed non-TinyML approaches, achieving higher accuracy and lower false alarm rates.
Sicari et al. [7]	Real-world smart home data (electricity consumption)	MQTT-based publish and subscribe protocol, AUPS, sticky policies	Raspberry Pi (4 NOS instances), MQTT brokers (2–10)	Performance: CPU load on brokers decreased with more brokers; latency reduced; CPU load on NOSs remained constant.	Proposed a security-aware IoT platform using fog computing with dual-layer architecture (NOSs and brokers). Increasing the number of brokers improved load balancing and reduced latency, enhancing system efficiency.
Hanaa Attou et al. [8]	NSL-KDD, Bot-IoT	Cloud-based		NSL-KDD: Accuracy = 98.3 %; Precision = 96.3 %; Recall = 46.0 % Bot-IoT: Accuracy = 99.99 %; Precision = 100 %; Recall = 100 %	Proposed a cloud-based intrusion detection model using RF and feature engineering. Highlights the effectiveness of RF with reduced feature sets for cloud intrusion detection.

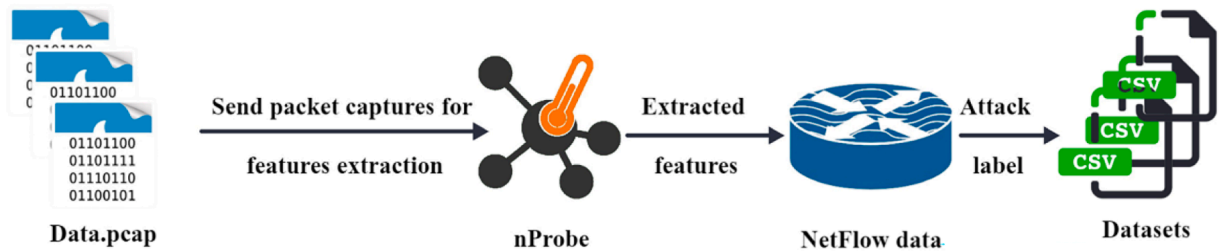


Fig. 1. The feature extraction process.

$$Kappa = \frac{p_0 - p_e}{1 - p_e} \quad (2)$$

where TP denotes true positives, FP signifies false positives, TN signifies true negatives, and FN signifies false negatives, and p_0 represents the empirical probability of agreement on the label assigned to any sample (the observed agreement ratio), and p_e signifies the expected agreement if both annotators indiscriminately assign labels.

The selected metrics were chosen because the goal of this study is to find a dependable and robust model for identifying types of attacks. These metrics are not affected by imbalanced data and can identify the

most stable and effective models [43]. As shown in Fig. 2. b and c, the NF-ToN-IoT-v2 and NF-BoT-IoT-v2 datasets contain highly imbalanced classes. Consequently, traditional machine learning metrics, such as accuracy, recall, precision, and F1-score, are inadequate for representing the most efficient and robust model. This is because the metrics commonly used in previous studies [44–50] are highly sensitive to imbalanced data. To assess the estimators for each boosting technique and to avoid overfitting during model selection, five-fold cross-validation was employed in all empirical evaluations.

To evaluate the performance of TinyML models, we incorporated three key metrics tailored for microcontrollers: SRAM usage, Flash

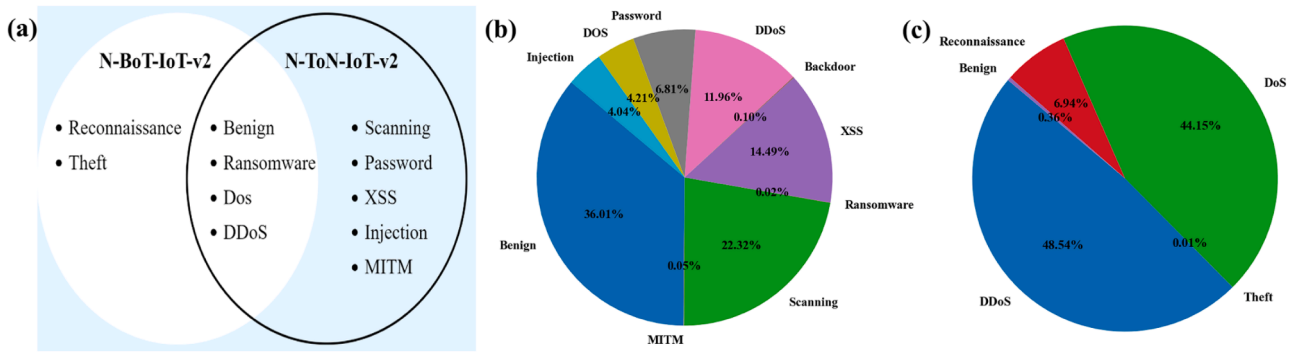


Fig. 2. (a) Predefined classes of the two datasets; (b) Percentage of classes for NF-ToN-IoT-v2; (c) Percentage of classes for NF-Bot-IoT-v2.

memory usage, and latency. These metrics can provide valuable insights into the energy consumption and computational efficiency of the models [51,52], which are critical for deployment on resource-constrained IoT devices:

- **Static Random-Access Memory (SRAM):** These metric gauges the utilization of static random-access memory by a TinyML model on a microcontroller [30]. It quantifies the amount of SRAM consumed during model execution, offering crucial insights into memory efficiency and usability for deployment in devices with limited resources.
- **Flash Memory (FM):** FM measures the utilization of flash memory using a TinyML model on a microcontroller [29]. This metric is vital for assessing the storage requirements of the model and its compatibility with devices with a restricted storage capacity.
- **Latency (Prediction Latency):** In our study, the latency for TinyML models is computed as the average time taken to make a prediction across five samples. Specifically, it involves measuring the duration from the input data to the prediction output [28]. This metric provides an indication of the responsiveness and operational speed of the model in real-world applications on constrained devices.

The TinyML models with higher Flash memory and SRAM usage, along with increased latency, were associated with higher energy consumption [51,52].

4.3. Statistical tests and ranking methods

- **Scott-Knott (SK):** Test: Researchers utilized the SK test to simultaneously determine the statistical significance of differences among multiple ML techniques. Developed in 1974 by Scott and Knott [31], this clustering algorithm is distinct from other statistical cluster analysis methods, such as Tukey and Student-Newman-Keuls (SNK) tests, because it classifies techniques into non-overlapping groups [53], thereby ensuring clear classifications. The SK test is well-regarded in numerous studies on the selection of ensemble methods [54,55]. In this investigation, the SK test was employed for clustering, ranking, and identifying substantial differences between the ensemble techniques in terms of kappa. Each experiment described in this paper was performed using a 5-fold validation method.
- **Borda count (BC):** is a voting method used to determine a single winner. In this method, voters rank candidates according to their preferences. Each candidate receives points based on their ranking, with the lowest-ranked candidate receiving the fewest. The total points received by each candidate are calculated, and the candidate with the highest total score is declared the winner [32]. In this study, we employed the BC method to identify the model with the best performance when all models were considered equally important.

4.4. Hyperparameter selection strategy

Due to the large size of NetFlow IoT IDS datasets and the constraints of execution time and computational resources, we restricted parameter selection to six values, as shown in Table 2. This limitation applied whether a single parameter was used or two parameters were considered (two values of first parameter and three possible values of second value). The selection of these hyperparameters for each ML technique is as follows:

- **For singular learning techniques:** (1) regarding DT, the maximum depth parameter was set between 4 and 42. This range was chosen to balance underfitting and overfitting, considering the 52 features available [56] after dummy coding (with a maximum depth of 42) and the trade-off between accuracy and resource efficiency during deployment [56]. Other parameters, such as the maximum number of leaf nodes (which restricts excessive node expansion) and the information gain threshold for splits, are also crucial for optimizing performance. However, we limited the depth to three to achieve a balance between accuracy and memory usage on resource-constrained MCUs. (2) for NB, the variance smoothing hyperparameter (*var_smoothing*) was set within a logarithmic range of $1e^{-09}$ to $1e^{-04}$. This range was chosen to ensure stable performance across datasets with different noise levels and sparsity. Increasing *var_smoothing* helps prevent division by zero for features with low variance; while decreasing it too much can lead to overfitting when features are highly noisy. Based on previous studies [57, 58], we selected this range as a trade-off to balance these effects and optimize model performance. (3) For the MLP algorithm, the hidden layer sizes were set to {6, 16, 26, 36, 46} to balance model complexity and resource efficiency. Smaller networks are well-suited for simpler datasets, while larger networks can capture more complex patterns but may require higher computational resources. This range was chosen to identify the optimal configuration for deployment on embedded devices.
- **For ensemble learning methods:** the number of estimators was set to {2, 4, 6} to control the number of trees in the ensemble. This narrow range was chosen to reduce computational cost, training time, and, most importantly, memory usage (SRAM and flash

Table 2
Hyperparameters used for ML techniques.

Type	Algorithm	Hyperparameters
Singular	DT	'max_depth': {4, 12, 20, 28, 36, 42}.
	NB	'var_smoothing': {1e-09, 1e-08, 1e-07, 1e-06, 1e-05, 1e-04}
Ensemble	MLP	'hidden_layer_sizes': {6, 16, 26, 36, 46}.
	RF	'estimators': [{2,4,6}] and 'max_depth': [{8,16}]
	ET	
	XGB	

memory) during deployment. The maximum depth parameter was set to {8, 16} to limit tree depth, particularly for (1) RF and ET which these methods rely on parallel training and bootstrap aggregation using deep trees based on Scikit-Learn's default DT settings. Optimizing tree depth is essential to prevent deployment errors due to excessive memory usage, especially on resource-constrained MCUs such as Arduino. Additionally, the number of features used for different thresholds was set to {13, 17, 20, 33}, ensuring that the tree depth of the DT estimators in ensemble methods remains within an acceptable range. (2) XGBoost is a boosting technique that constructs a strong classifier by sequentially combining multiple weak classifiers, typically decision stumps (shallow one-level decision trees). Each new model in the boosting process corrects errors from the previous ones using weak learners. Since tree depth is inherently optimized in boosting, further tuning is generally unnecessary. However, the parameters were set to match those of RF and ET to maintain consistency within the other ensemble learning techniques used.

4.5. Methodology

This study presents a novel approach for evaluating TinyML models on the Arduino UNO, a highly resource-constrained embedded device, to generalize the findings for other IoT devices in IoT-IDS scenarios. It examines the effects of three singular ML techniques (NB, DT, and MLP) and three ensemble ML techniques (ET, RF, and XGB) under various feature thresholds. The performance of these models is evaluated using two robust metrics for handling imbalanced data, along with three additional metrics specifically tailored for assessing models on embedded devices. This approach aims to investigate how the feature thresholds and the hyperparameters of each ML technique influence the

performance of the TinyML models on the NF-ToN-IoT-v2 and NF-BoT-IoT-v2 datasets on the Arduino UNO.

The techniques shown in Fig. 3 were selected to comprehensively evaluate TinyML models for intrusion detection in IoT environments, addressing challenges such as the heterogeneous nature of IoT data, imbalanced class distributions, and the need for robust classification models. Filter-based FS methods, including ANOVA for numerical features and mRMR for categorical features, were applied to ensure an effective selection process. The ML techniques were chosen for their feasibility on Arduino UNO, enabling efficient deployment in resource-constrained environments, while ensemble methods were selected for their ability to enhance classification performance, particularly in handling imbalanced datasets. To ensure model reliability, Kappa and MCC were used as performance metrics due to their robustness against class imbalance, while SRAM, FM, and latency were considered to assess computational efficiency and resource utilization during deployment. Additionally, cross-validation and statistical tests such as SK were applied to ensure consistent model performance across all datasets, providing a well-rounded evaluation of TinyML-based intrusion detection systems.

As shown in Fig. 3, the experimental setup consists of Steps 1–3, 5, and 6, which include data preprocessing, feature engineering, model training, validation, and model conversion, all conducted on the Toubkal supercomputer at UM6P [59]. Step 4, involving model deployment and evaluation, was performed on the Arduino UNO. The detailed steps are as follows:

- **Step 1:** Prepare the raw data for analysis by performing several cleaning tasks. This includes removing missing values, duplicates, and irrelevant information. Categorical features with more than

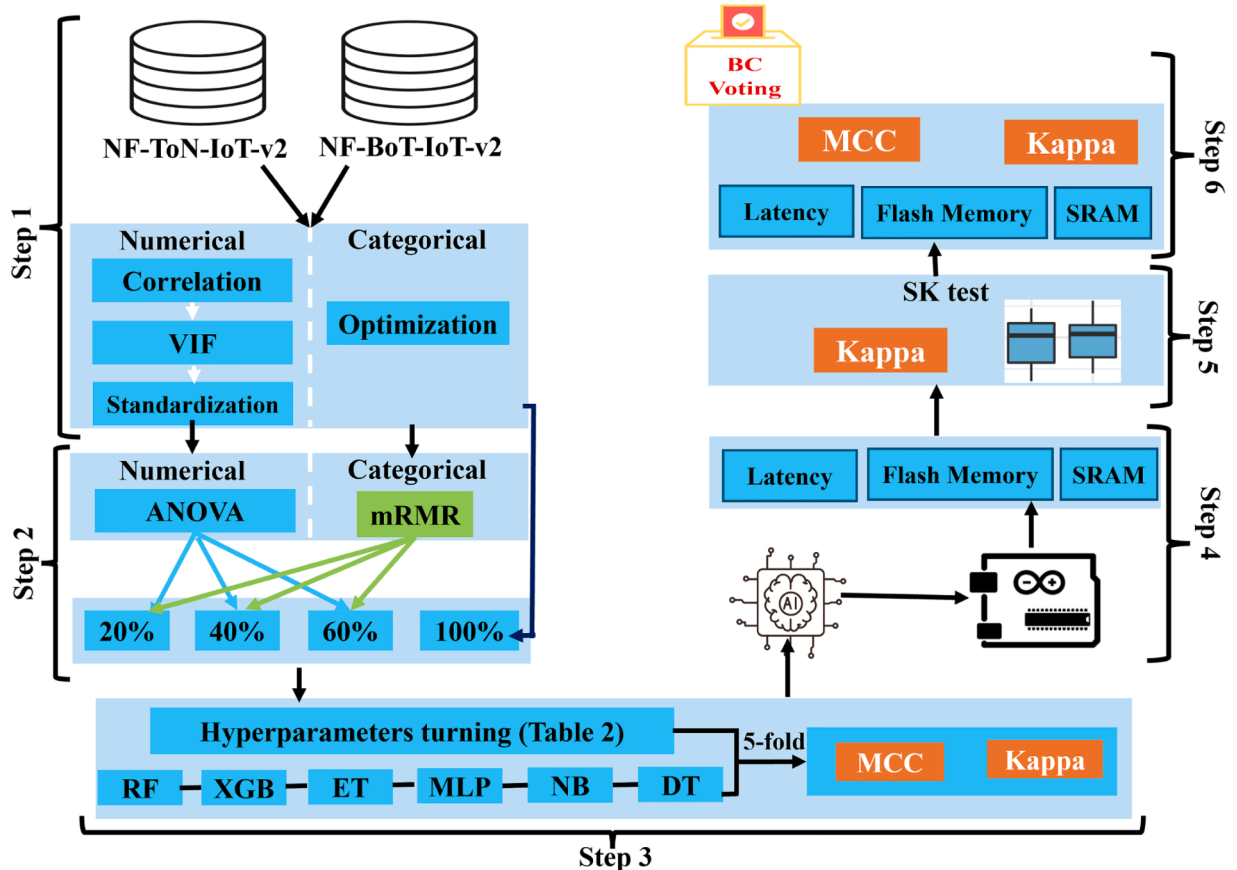


Fig. 3. Experimental design.

eight classes were simplified by reducing the number of classes. Numerical attributes were normalized using Eq. (3):

$$Z = \frac{x_i - \mu}{\sigma} \quad (3)$$

Where μ represents the mean, x_i is an individual data point, and σ is the standard deviation. In addition, numerical features with a Pearson correlation coefficient greater than 95 % and a variance inflation factor (VIF) below five were selectively removed in pairs [60].

• **Step 2:** Apply two FS techniques (mRMR for categorical features and ANOVA for numerical features) along with four feature thresholds (20 %, 40 %, 60 %, and 100 %). This process results in eight variations of the dataset (8 dataset variations = 2 NetFlow IoT datasets $\times$ 4 thresholds).

• **Step 3:** Develop and evaluate 288 classifiers (288 = (3 singular ML techniques + 3 ensemble ML techniques) $\times$ 6 hyperparameter combinations $\times$ 8 datasets from Step 2). These classifiers were constructed using the Toubkal supercomputer and built with three singular methods and three ensemble techniques, each tested with six different hyperparameter configurations, as shown in Table 2. Five-fold cross-validation was applied, and performance was evaluated using two prediction metrics (MCC and Kappa).

• **Step 4:** Save the 288 classifiers developed in Step 3 and convert them from Python to C to generate TinyML models. This conversion is performed using Python libraries designed for TinyML development, specifically: (1) MicroMLGen for DT, RF, XGB, and NB, and (2) EMLearn for ET and MLP. The generated models are first exported in a development environment (supercomputer) and then deployed on an Arduino UNO. Deployment involves integrating the exported header file (.h) and coding the corresponding C file to complete the implementation. Once deployed, the models are evaluated for SRAM and Flash Memory (FM) usage. Additionally, test instances are provided as input to measure prediction latency and assess overall performance.

• **Step 5:** Collect and export deployment metrics related to the embedded device, including FM, SRAM, and latency, from the Arduino UNO. These results are then combined with performance metrics (MCC and Kappa) for each ML technique, hyperparameter combination, feature threshold, and dataset. The SK test is then applied to group models based on the Kappa criterion, which serves as the primary metric for statistically assessing significant differences in classifier performance.

• **Step 6:** Compare the top-performing TinyML models from the SK test clusters obtained in Step 5, for each feature threshold and dataset. This comparison is done using the BC voting system, which is based on (1) Kappa and MCC metrics, with scores ranked from highest to lowest for each prediction metric; and (2) latency, SRAM, and FM metrics, ranked from lowest to highest. In this study, the BC system assigns the following weights to each metric: 2.5 for MCC and Kappa (model accuracy), 2 for latency (to ensure real-time performance), 1.5 for SRAM (due to its impact on battery life and device autonomy), and 1 for FM.

4.6. Abbreviation

To enhance readability and simplify model names, this paper uses specific naming conventions for the models, as follows:

MLTechnique.FeatureThreshold.HyperparametersValues

The abbreviations for the hyperparameters are as follows: D for max tree depth, S for smoothing variable, L for hidden layer sizes, and E for number of estimators. For example, MLP20_L6 indicates the MLP technique using 20 % of features and a hidden layer size of 6.

5. Results and discussion

This section presents and examines the results obtained by testing three TinyML single classifiers (DT, NB, and MLP) and three TinyML ensemble classifiers (RF, XGB, and ET), each with six parameter combinations, as listed in Table 2. We used two feature selection techniques (ANOVA and mRMR) and four feature thresholds (20 %, 40 %, 60 %, and 100 %) across the NF-BoT-IoT-v2 and NF-ToN-IoT-v2 datasets. The findings are discussed in relation to the RQs outlined in Section 1.

5.1. Impact of hyperparameters on the efficacy of TinyML singular models on Arduino UNO (RQ1)

This section examines the impact of hyperparameters and feature numbers on Arduino UNO metrics for singular ML techniques (MLP, NB, and DT). We converted 144 variants (48 per singular ML technique) generated in the Toubkal supercomputer of UM6P [59] into TinyML models using (1) the MicroMLGen library for DT and NB, and (2) the EMLearn library for MLP. These models were deployed on Arduino UNO device. We measured SRAM and FM in bytes and latency in microseconds, with latency calculated as the average prediction time for five samples for each TinyML singular model. Tables 3 and 4 show the Arduino UNO performance metrics for deployed models for NF-ToN-IoT-v2 and NF-BoT-IoT-v2, respectively, given the memory constraints of the Arduino UNO. Additionally, the results of all generated models (deployed and not deployed) are shown in Tables A.1 and A.2 of Appendix A for NF-ToN-IoT-v2 and NF-BoT-IoT-v2, respectively.

Table 3 displays the deployed TinyML singular models on Arduino UNO generated by MLP, NB, and DT over the NF-ToN-IoT-v2 dataset and the four thresholds (20 %, 40 %, 60 %, and 100 %) obtained from ANOVA and mRMR. From Table 3, we observe that:

- For MLP using the six hyperparameters (number of neurons in hidden layer = {L6, L16, L26, L36, L46, L56}), the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, the six generated models with constant SRAM of 6410 bytes, constant FM of 526 bytes, and constant latency of 9.6 μ s; (2) using 40 % of the features, the six generated models with constant SRAM of 6690 bytes, constant FM of 806 bytes, and constant latency of 9.6 μ s; (3) using 60 % of the features, the six generated models with constant SRAM of 7010 bytes, constant FM of 1112 bytes, and constant latency of 9.6 μ s; (4) using all features, the six generated models with constant SRAM of 7350 bytes, constant FM of 1466 bytes, and constant latency of 9.6 μ s.
- For NB using the six hyperparameters (smoothing = {S10⁻¹, S10⁻², S10⁻³, S10⁻⁴, S10⁻⁵, S10⁻⁶, S10⁻⁷}), the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, the six generated models with constant SRAM of 8546 bytes, constant FM of 512 bytes, and latency ranging from 32,445.6 to 33,359.2 μ s; (2) using 40 % of the features, the six generated models with constant SRAM of 13,600 bytes, constant FM of 792 bytes, and latency ranging from 56,776.8 to 57,605.6 μ s; (3) using 60 % of the features, the six generated models with constant SRAM of 14,280 bytes, constant FM of 1112 bytes, and latency ranging from 67,804 to 71,159.2 μ s; (4) using all features, the six generated models with constant SRAM of 20,180 bytes, constant FM of 1452 bytes, and latency ranging from 91,724.8 to 93,665.6 μ s.
- For DT using the six tree depth parameters (D4, D12, D20, D28, D36, D40), the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, DT20_D4 and DT20_D12 models with constant FM of 512 bytes, SRAM of 32,256 and 22,934 bytes, respectively, and latency of 23.2 and 48 μ s, respectively; (2) using 40 % of the features, only the DT40_D4 model with FM of 792 bytes, SRAM of 32,256 bytes, and latency of 23.2 μ s; (3) using 60 % of the features, only the DT60_D4 model with FM of 1112 bytes, SRAM of 4558 bytes, and latency of 22.4 μ s; (4) using all features, only the

Table 3

Performance of deployed TinyML singular models on Arduino Uno over the NF-ToN-IoT-v2 dataset.

NB (MicroMLGen)				MLP (EMLearn)				DT (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
MLP20_L6	9.6	6410	526	NB20_S1e-04	32,445.6	8546	512	DT20_D4	23.2	32,256	512
MLP20_L16	9.6	6410	526	NB20_S1e-05	32,654.4	8546	512	DT20_D12	48	22,934	512
MLP20_L26	9.6	6410	526	NB20_S1e-06	33,359.2	8546	512				
MLP20_L36	9.6	6410	526	NB20_S1e-07	32,537.6	8546	512				
MLP20_L46	9.6	6410	526	NB20_S1e-08	32,705.6	8546	512				
MLP20_L56	9.6	6410	526	NB20_S1e-09	33,190.4	8546	512				
MLP40_L6	9.6	6690	806	NB40_S1e-04	56,776.8	13,600	792	DT40_D4	23.2	32,256	792
MLP40_L16	9.6	6690	806	NB40_S1e-05	56,982.4	13,600	792				
MLP40_L26	9.6	6690	806	NB40_S1e-06	57,830.4	13,600	792				
MLP40_L36	9.6	6690	806	NB40_S1e-07	56,798.4	13,600	792				
MLP40_L46	9.6	6690	806	NB40_S1e-08	57,035.2	13,600	792				
MLP40_L56	9.6	6690	806	NB40_S1e-09	57,605.6	13,600	792				
MLP60_L6	9.6	7010	1126	NB60_S1e-04	70,678.4	14,248	1112	DT60_D4	22.4	4558	1112
MLP60_L16	9.6	7010	1126	NB60_S1e-05	66,005.6	14,248	1112				
MLP60_L26	9.6	7010	1126	NB60_S1e-06	68,681.6	14,248	1112				
MLP60_L36	9.6	7010	1126	NB60_S1e-07	71,159.2	14,280	1112				
MLP60_L46	9.6	7010	1126	NB60_S1e-08	68,646.4	14,280	1112				
MLP60_L56	9.6	7010	1126	NB60_S1e-09	67,804	14,408	1112				
MLP100_L6	9.6	7350	1466	NB100_S1e-04	91,724.8	20,180	1452	DT100_D4	23.2	4854	1452
MLP100_L16	9.6	7350	1466	NB100_S1e-05	92,131.2	20,180	1452				
MLP100_L26	9.6	7350	1466	NB100_S1e-06	93,553.6	20,180	1452				
MLP100_L36	9.6	7350	1466	NB100_S1e-07	92,084.8	20,180	1452				
MLP100_L46	9.6	7350	1466	NB100_S1e-08	92,372.8	20,180	1452				
MLP100_L56	9.6	7350	1466	NB100_S1e-09	93,665.6	20,180	1452				

Table 4

Performance of deployed TinyML singular models on Arduino Uno over the NF-BoT-IoT-v2 dataset.

MLP (EMLearn)				NB (MicroMLGen)				DT (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
MLP20_L6	9.6	6330	446	NB20_S1e-04	12,723.2	6488	432	DT20_D4	24	3694	432
MLP20_L16	8.8	6330	446	NB20_S1e-05	12,771.2	6488	432	DT20_D12	50.4	13,896	432
MLP20_L26	9.6	6330	446	NB20_S1e-06	12,812	6488	432				
MLP20_L36	9.6	6330	446	NB20_S1e-07	12,719.2	6488	432				
MLP20_L46	8.8	6330	446	NB20_S1e-08	12,732	6488	432				
MLP20_L56	9.6	6330	446	NB20_S1e-09	12,804.8	6488	432				
MLP40_L6	9.6	6550	666	NB40_S1e-04	21,900.8	8472	652	DT40_D4	24	3918	652
MLP40_L16	9.6	6550	666	NB40_S1e-05	21,964	8472	652	DT40_D12	50.4	16,480	652
MLP40_L26	9.6	6550	666	NB40_S1e-06	22,090.4	8472	652				
MLP40_L36	9.6	6550	666	NB40_S1e-07	21,886.4	8472	652				
MLP40_L46	9.6	6550	666	NB40_S1e-08	21,904.8	8472	652				
MLP40_L56	9.6	6550	666	NB40_S1e-09	22,048.8	8472	652				
MLP60_L6	9.6	6768	886	NB60_S1e-04	31,448.8	10,694	872	DT60_D4	24	4146	872
MLP60_L16	9.6	6768	886	NB60_S1e-05	31,514.4	10,694	872	DT60_D12	50.4	23,044	872
MLP60_L26	9.6	6768	886	NB60_S1e-06	31,737.6	10,694	872				
MLP60_L36	9.6	6768	886	NB60_S1e-07	31,476.8	10,694	872				
MLP60_L46	9.6	6768	886	NB60_S1e-08	31,521.6	10,694	872				
MLP60_L56	9.6	6768	886	NB60_S1e-09	31,732	10,694	872				
MLP100_L6	9.6	7070	1186	NB100_S1e-04	31,736	9188	1172	DT100_D4	17.6	4444	1172
MLP100_L16	9.6	7070	1186	NB100_S1e-05	29,991.2	9188	1172	DT100_D12	33.6	23,416	1172
MLP100_L26	9.6	7070	1186	NB100_S1e-06	29,443.2	9188	1172				
MLP100_L36	9.6	7070	1186	NB100_S1e-07	32,092	9188	1172				
MLP100_L46	9.6	7070	1186	NB100_S1e-08	29,943.2	9208	1172				
MLP100_L56	9.6	7070	1186	NB100_S1e-09	30,190.4	9208	1172				

DT100_D4 model with FM of 1452 bytes, SRAM of 4854 bytes, and latency of 23.2 μ s.

Table 4 displays the deployed TinyML singular models on Arduino UNO generated by MLP, NB, and DT over the NF-BoT-IoT-v2 dataset and the four thresholds (20 %, 40 %, 60 %, and 100 %) obtained from ANOVA and mRMR. From Table 4, we observe that:

- For MLP using the six hyperparameters (number of neurons in hidden layer = {L6, L16, L26, L36, L46, L56}), the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, the six generated models with constant SRAM of 6330 bytes, constant FM of 446 bytes, and latency ranging from 8.8 to 9.6 μ s; (2) using 40 % of

the features, the six generated models with constant SRAM of 6550 bytes, constant FM of 666 bytes, and constant latency of 9.6 μ s; (3) using 60 % of the features, the six generated models with constant SRAM of 6768 bytes, constant FM of 886 bytes, and constant latency of 9.6 μ s; and (4) using all features, the six generated models with constant SRAM of 7070 bytes, constant FM of 1186 bytes, and constant latency of 9.6 μ s.

- For NB using the six hyperparameters (smoothing = { $S10^{-1}$, $S10^{-2}$, $S10^{-3}$, $S10^{-4}$, $S10^{-5}$, $S10^{-6}$, $S10^{-7}$ }), the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, the six generated models with constant SRAM of 6488 bytes, constant FM of 432 bytes, and latency ranging from 12,719.2 to 12,812 μ s; (2) using 40 % of the features, the six generated models with constant SRAM of 8472

bytes, constant FM of 652 bytes, and latency ranging from 21,886.4 to 22,090.4 μ s; (3) using 60 % of the features, the six generated models with constant SRAM of 10,694 bytes, constant FM of 872 bytes, and latency ranging from 31,448.8 to 31,737.6 μ s; and (4) using all features, the six generated models with constant SRAM of 9188 bytes, constant FM of 1172 bytes, and latency ranging from 29,443.2 to 32,092 μ s.

- For DT using the six tree depth parameters (D4, D12, D20, D28, D36, D40), the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, DT20_D4 and DT20_D12 models with constant FM of 432 bytes, SRAM of 3694 and 13,896 bytes, respectively, and latency of 24 and 50.4 μ s, respectively; (2) using 40 % of the features, DT40_D4 and DT40_D12 models with constant FM of 652 bytes, SRAM of 3918 and 16,480 bytes, respectively, and latency of 24 and 50.4 μ s, respectively; and (3) using 60 % of the features, DT60_D4 and DT60_D12 models with constant FM of 872 bytes, SRAM of 4146 and 23,044 bytes, respectively, and latency of 24 and 50.4 μ s, respectively; (4) using all features, DT100_D4 and DT100_D12 models with constant FM of 1172 bytes, SRAM of 4444 and 23,416 bytes, respectively, and latency of 17.6 and 33.6 μ s, respectively.

In summary, for the singular ML techniques used, increasing the number of features increases the usage of SRAM and FM. For MLP and NB, all generated models using the two datasets, the four thresholds, and the different hyperparameters are supported by the Arduino UNO. Increasing the number of neurons in the hidden layer for MLP and the smoothing value for NB does not affect the increase in the SRAM, FM, and latency. In contrast, the models generated by DT are not all supported. Specifically, (1) Using NF-ToN-IoT-v2, only the models with a tree depth of 4 and 12 for 20 % of the features (DT_20_D4 and DT_20_D12) and the models with a tree depth of 4 using 40 %, 60 %, and 100 % of the features (DT_40_D4, DT_60_D4, and DT_100_D4) are supported by the Arduino UNO. (2) Using TF-BoT-IoT-v2, the models generated with tree depths of 4 and 12 are supported for all four feature thresholds (DT_20_D4, DT_20_D12, DT_40_D4, DT_40_D12, DT_60_D4, DT_60_D12, DT_100_D4, and DT_100_D12), indicating that increasing the tree depth increases the usage of SRAM and FM. This highlights the importance of FS and hyperparameter tuning in optimizing the deployment of TinyML models on resource-constrained devices like Arduino UNO. Furthermore, these supported TinyML models can be deployed on other embedded devices in the IoT field, as they have been supported by the Arduino UNO.

5.2. Impact of hyperparameters on the efficacy of TinyML ensemble models on Arduino UNO (RQ2)

This section examines the impact of the hyperparameters and feature numbers on the Arduino UNO metrics for ensemble ML techniques (ET, RF, and XGB). We converted 144 variants (48 per ensemble ML technique), generated using the Toubkal supercomputer at UM6P, into TinyML models. The conversion utilized the MicroMLGen library for RF and XGB, and the EMLearn library for ET. These models were deployed on the Arduino UNO device. We measured SRAM and FM in bytes and latency in microseconds, with latency calculated as the average prediction time for five samples for each TinyML singular model. Tables 5 and 6 show the Arduino UNO performance metrics for the deployed models for NF-ToN-IoT-v2 and NF-BoT-IoT-v2, respectively, considering the memory constraints of the Arduino UNO. Additionally, the results of all generated models (both deployed and non-deployed) are shown in Tables A.3 and A.4 of Appendix A for NF-ToN-IoT-v2 and NF-BoT-IoT-v2, respectively.

Table 5 displays the deployed TinyML ensemble models on Arduino UNO generated by ET, RF, and XGB using the six estimators and tree depth combinations (E2D8, E2D16, E4D8, E4D16, E6D8, E6D16) over the NF-ToN-IoT-v2 dataset and the four thresholds obtained from ANOVA and mRMR. From Table 5, we observe that:

- For ET, the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, five generated models (ET20_E2D8, ET20_E2D16, ET20_E4D8, ET20_E4D16, and ET20_E6D8) with constant SRAM of 4770 bytes, constant FM of 393 bytes, and constant latency of 8 μ s; (2) using 40 % of the features, four generated models (ET40_E2D8, ET40_E2D16, ET40_E4D8, and ET40_E6D8) with constant SRAM of 4910 bytes, constant FM of 533 bytes, and constant latency of 8 μ s; (3) using 60 % of the features, three generated models (ET60_E2D8, ET60_E4D8, and ET60_E6D8) with constant SRAM of 5240 bytes, constant FM of 863 bytes, and constant latency of 8 μ s; (4) using 100 % of the features, four generated models (ET100_E2D8, ET100_E2D16, ET100_E4D8, and ET100_E6D8) with constant SRAM of 5240 bytes, constant FM of 863 bytes, and constant latency of 8 μ s.
- For RF, the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, three generated models (RF20_E2D8, RF20_E4D8, and RF20_E6D8) with constant FM of 512 bytes, SRAM ranging from 9526 to 19,164 bytes, and latency ranging from 23.2 to 219.2 μ s; (2) using 40 % of the features, three generated models (RF40_E2D8, RF40_E4D8, and RF40_E6D8) with constant FM of 792 bytes, SRAM ranging from 12,834 to 30,828 bytes, and latency ranging from 128 to 243.2 μ s; (3) using 60 % of the features, RF60_E2D8 and RF60_E4D8 models with constant FM of 1112 bytes,

Table 5
Performance of deployed TinyML ensemble models on Arduino Uno over the NF-ToN-IoT-v2 dataset.

ET (EMLearn)				RF (MicroMLGen)				XGB (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
ET20_E2D8	8	4770	393	RF20_E2D8	23.2	9526	1536				
ET20_E2D16	8	4770	393	RF20_E4D8	171.2	15,176	512				
ET20_E4D8	8	4770	393	RF20_E6D8	219.2	19,164	512				
ET20_E4D16	8	4770	393								
ET20_E6D8	8	4770	393								
ET40_E2D8	8	4910	533	RF40_E2D8	128	12,834	792				
ET40_E2D16	8	4910	533	RF40_E4D8	180.8	20,970	792				
ET40_E4D8	8	4910	533	RF40_E6D8	243.2	30,828	792				
ET40_E6D8	8	4910	533								
ET60_E2D8	8	5070	693	RF60_E2D8	134.4	14,154	1112				
ET60_E4D8	8	5070	693	RF60_E4D8	200	25,288	1112				
ET60_E6D8	8	5070	693								
ET100_E2D8	8	5240	863	RF100_E2D8	124.8	16,002	1452				
ET100_E2D16	8	5240	863	RF100_E4D8	192	28,406	1452				
ET100_E4D8	8	5240	863								
ET100_E6D8	8	5240	863								

Table 6

Performance of deployed TinyML ensemble models on Arduino Uno over the NF-BoT-IoT-v2 dataset.

ET (EMLearn)				RF (MicroMLGen)				XGB (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
ET20_E2D8	8	4730	353	RF20_E2D8	91.2	7808	432	XGB20_E2D8	332	31,162	432
ET20_E2D16	8	4730	353	RF20_E4D8	142.6	12,948	432				
ET20_E4D8	8	4730	353	RF20_E6D8	197.6	17,440	432				
ET20_E6D8	8	4730	353								
ET40_E2D8	8	4840	463	RF40_E2D8	92	7618	652				
ET40_E2D16	8	4840	463	RF40_E4D8	144	12,096	652				
ET40_E4D8	8	4840	463	RF40_E6D8	201.6	16,556	652				
ET40_E6D8	8	4840	463								
ET60_E2D8	8	4950	573	RF60_E2D8	97.6	9390	872				
ET60_E2D16	8	4950	573	RF60_E4D8	156	14,426	872				
ET60_E4D8	8	4950	573	RF60_E6D8	215.2	20,380	872				
ET60_E6D8	8	4950	573								
ET100_E2D16	8	5100	723	RF100_E2D8	98.4	9760	1172				
ET100_E4D8	8	5100	723	RF100_E4D8	159.2	16,032	1172				
ET100_E6D8	8	5100	723	RF100_E6D8	220	21,858	1172				

SRAM of 14,154 and 25,288 bytes, respectively, and latency of 134.4 and 200 μ s, respectively; (4) using all features, RF100_E2D8 and RF100_E4D8 models with constant FM of 1452 bytes, SRAM of 16,002 and 28,406 bytes, respectively, and latency of 124.8 and 192 μ s, respectively.

- For XGB, no TinyML model could be deployed on Arduino UNO due to insufficient memory.

Table 6 displays the deployed TinyML ensemble models on Arduino UNO generated by ET, RF, and DT using the six estimators and tree depth combinations (E2D8, E2D16, E4D8, E4D16, E6D8, E6D16) over the NF-ToN-IoT-v2 dataset and the four thresholds obtained from ANOVA and mRMR. From Table 6, we observe that:

- For ET, the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, four generated models (ET20_E2D8, ET20_E2D16, ET20_E4D8, and ET20_E6D8) with constant SRAM of 4730 bytes, constant FM of 353 bytes, and constant latency of 8 μ s; (2) using 40 % of the features, four generated models (ET40_E2D8, ET40_E2D16, ET40_E4D8, and ET40_E6D8) with constant SRAM of 4840 bytes, constant FM of 463 bytes, and constant latency of 8 μ s; (3) using 60 % of the features, four generated models (ET60_E2D8, ET60_E2D16, ET60_E4D8, and ET60_E6D8) with constant SRAM of 4950 bytes, constant FM of 573 bytes, and constant latency of 8 μ s; (4) using 100 % of the features, three generated models (ET100_E2D16, ET100_E4D8, and ET100_E6D8) with constant SRAM of 5100 bytes, constant FM of 723 bytes, and constant latency of 8 μ s.
- For RF, the TinyML models deployed on Arduino UNO are: (1) using 20 % of the features, three generated models (RF20_E2D8, RF20_E4D8, and RF20_E6D8) with constant FM of 432 bytes, SRAM ranging from 7808 to 17,440 bytes, and latency ranging from 95.2 to 197.6 μ s; (2) using 40 % of the features, three generated models (RF40_E2D8, RF40_E4D8, and RF40_E6D8) with constant FM of 652 bytes, SRAM ranging from 7618 to 16,556 bytes, and latency ranging from 92 to 201.6 μ s; (3) using 60 % of the features, three generated models (RF60_E2D8, RF60_E4D8, and RF60_E6D8) with constant FM of 872 bytes, SRAM ranging from 9390 to 20,380 bytes, and latency ranging from 97.6 to 215.2 μ s; (4) using all features, three generated models (RF100_E2D8, RF100_E4D8, and RF100_E6D8) with constant FM of 1172 bytes, SRAM ranging from 9760 to 21,858 bytes, and latency ranging from 98.4 to 220 μ s.
- For XGB, only the XGB20_E2D8 TinyML model with SRAM of 31,162 bytes, FM of 432 bytes, and latency of 332 μ s is deployable.

In summary, for the ensemble ML techniques used, increasing the number of features increases the usage of SRAM and FM. (1) For ET, the supported models have the same latency for all thresholds, and

increasing the number of estimators and tree depth does not increase the usage of SRAM and FM for each threshold. (2) For RF, the supported models for each threshold show that increasing the number of estimators and tree depth does not affect the increase in FM but does increase SRAM. (3) For XGB, only the XGB_20_E2D8 model is deployed over NF-BoT-IoT-v2 using a small number of estimators and a small tree depth (E2D8). In addition, these validated TinyML models can be deployed on other embedded devices in the IoT field, as they are compatible with the Arduino UNO.

5.3. Best model parameters, feature threshold, and FS combination for effective IDS construction with TinyML on Arduino UNO (RQ3)

In this section, we assess and compare the performance of the models supported by Arduino UNO using the two datasets and the four feature thresholds. The SK test based on Kappa metric was used to group models and identify the most effective SK clusters. Figs. 4–7 present the SK results of the combinations of singular and ensemble ML techniques and hyperparameters deployed on Arduino UNO for each threshold and NetFlow dataset. For instance, Fig. 4.a shows the statistical analysis using the SK test of the 22 models (22 = 6 MLP + 6 NB + 2 DT + 5 ET + 3 RF) deployed on Arduino UNO and retained from RQs 1 and 2 that utilize 20 % of the features.

For the NF-ToN-IoT-v2 dataset:

- From Fig. 4.a (20 % of the features), we observe the presence of six clusters. The first cluster contains three MLP models (MLP20_L56, MLP20_L46, and MLP20_L36), two RF models (RF20_E6D8 and RF20_E4D8), and one DT model (DT20_D12). The second cluster contains two MLP models (MLP20_L26 and MLP20_L16), two ET models (ET20_E4D16 and ET20_E2D16), and one RF model (RF20_E2D8). The third cluster contains one DT model (DT20_D4) and one MLP model (MLP20_L6). The fourth and fifth clusters contain two and one ET models, respectively (ET20_E6D8 and ET20_E4D8, and ET20_E2D8). The last cluster contains all six NB models.
- From Fig. 4.b (40 % of the features), we observe the presence of nine clusters. The first cluster contains five MLP models (MLP40_L56, MLP40_L46, MLP40_L36, MLP40_L26, and MLP40_L16). The second cluster contains three RF models (RF40_E6D8, RF40_E4D8, and RF40_E2D8) and one MLP model (MLP40_L6). The third and fifth clusters each contain one ET model (ET40_E2D16 and ET40_E6D8). The fourth cluster contains one DT model (DT40_D4). The sixth cluster contains two ET models (ET40_E2D8 and ET40_E4D8). The seventh, eighth, and ninth clusters contain one, two, and three NB models, respectively.

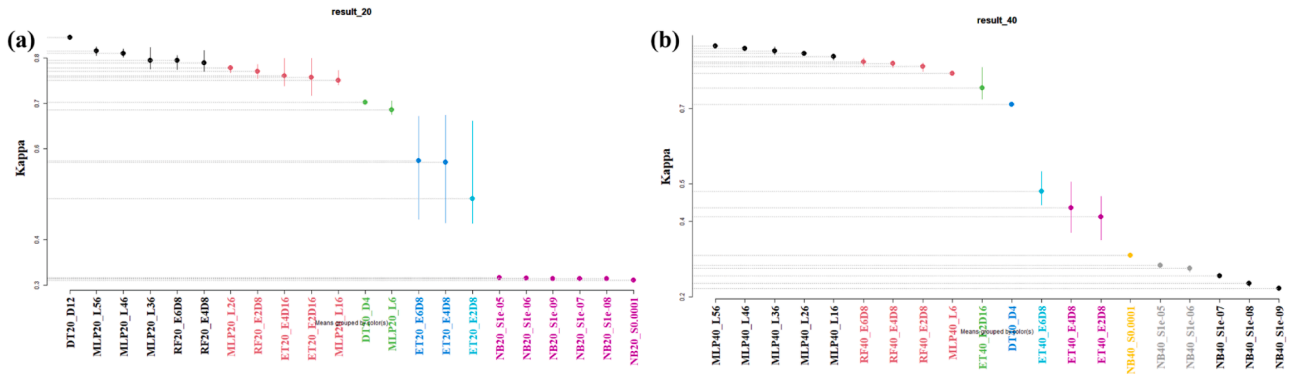


Fig. 4. SK test results for deployed TinyML models on Arduino UNO over NF-ToN-IoT-v2 using two thresholds: (a) 20 %; and (b) 40 % of features.

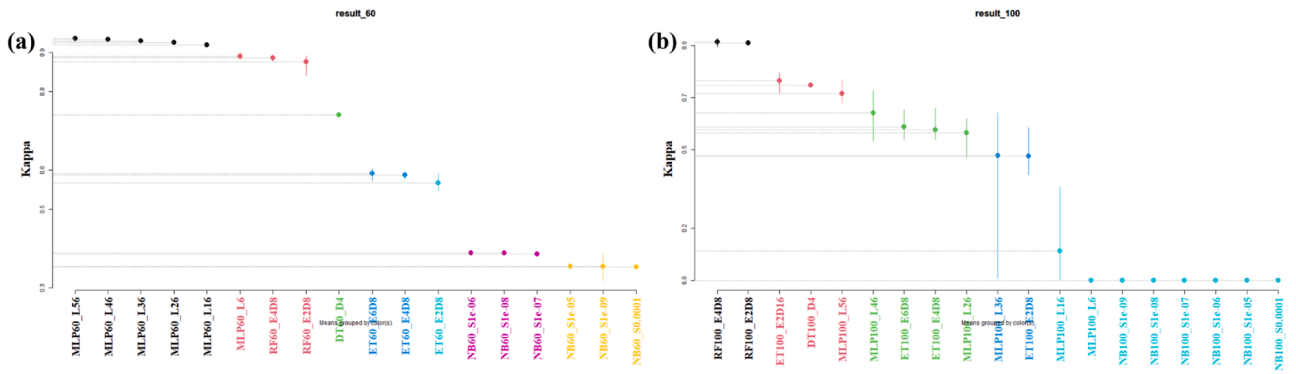


Fig. 5. SK test results for deployed TinyML models on Arduino UNO over NF-ToN-IoT-v2 using two thresholds: (a) 60 %; and (b) 100 % of features.

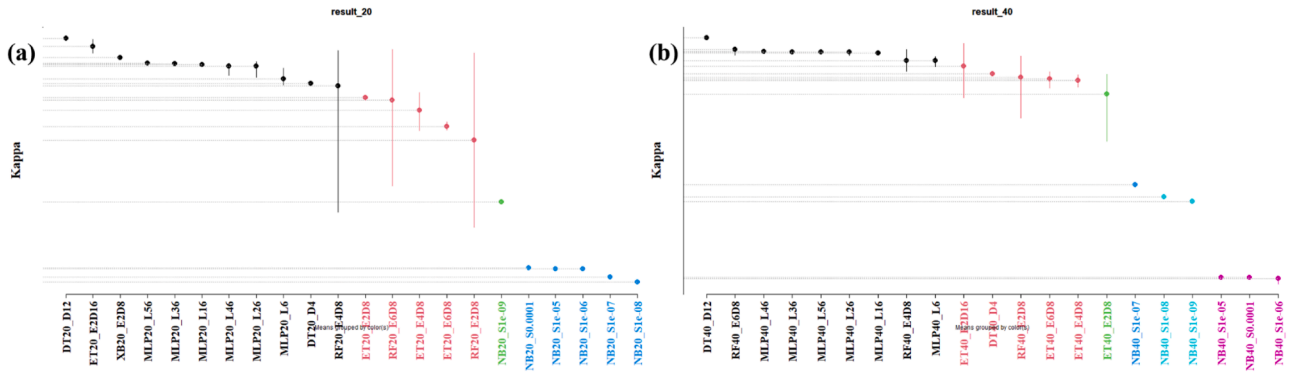


Fig. 6. SK test results for deployed TinyML models on Arduino UNO over NF-BoT-IoT-v2 using two thresholds: (a) 20 %; and (b) 40 % of features.

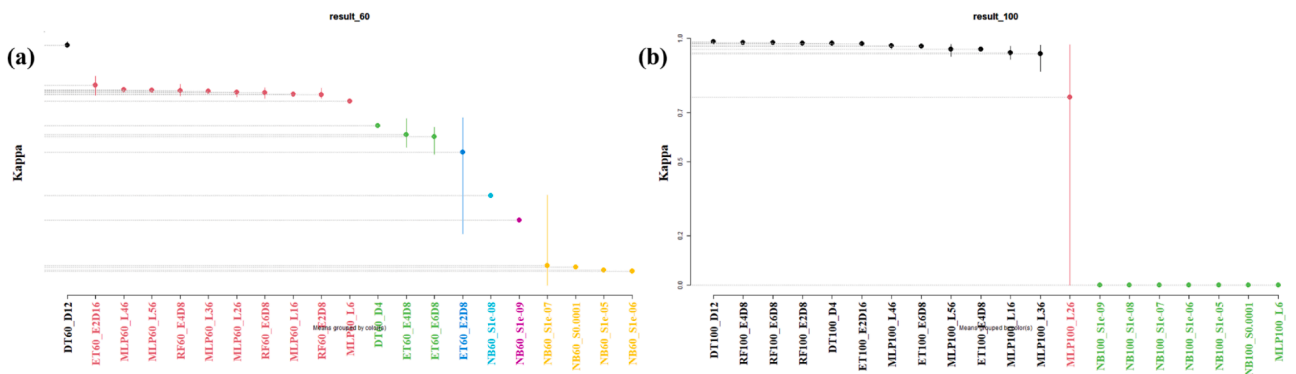


Fig. 7. SK test results for deployed TinyML models on Arduino UNO over NF-BoT-IoT-v2 using two thresholds: (a) 60 %; and (b) 100 % of features.

- From Fig. 5.a (using 60 % of the features), we observe the presence of seven clusters. The first cluster contains five MLP models (MLP60_L56, MLP60_L46, MLP60_L36, MLP60_L26, and MLP60_L16). The second cluster contains two RF models (RF60_E4D8 and RF60_E2D8) and one MLP model (MLP60_L6). The third cluster contains one DT model (DT60_D4). The fourth and fifth clusters contain two ET models (ET60_E6D8 and ET60_E4D8) and one ET model (ET60_E2D8), respectively. The sixth and seventh clusters each contain three NB models.
- From Fig. 5.b (100 % of the features), we observe the presence of five clusters. The first cluster contains two RF models (RF100_E4D8 and RF100_E2D8). The second cluster contains one ET model (ET100_E2D16), one DT model (DT100_D4), and one MLP model (MLP100_L56). The third cluster contains two MLP models (MLP100_L46 and MLP100_L26) and two ET models (ET100_E6D8 and ET100_E4D8). The fourth cluster contains one MLP model (MLP100_L36) and one ET model (ET100_E2D8). The last cluster contains two MLP models (MLP100_L16 and MLP100_L6) and all six NB models.

For the NF-BoT-IoT-v2 dataset:

- From Fig. 6.a (20 % of the features), we observe the presence of four clusters. The first cluster contains all six MLP models, two DT models (DT20_D12 and DT20_D4), and one model each of ET, RF, and XGB (ET20_E2D16, RF20_E4D8, and XGB20_E2D8). The second cluster contains three ET models (ET20_E2D8, ET20_E4D8, and ET20_E6D8) and two RF models (RF20_E6D8 and RF20_E2D8). The third and fourth clusters contain one and five NB models, respectively.
- From Fig. 6.b (40 % of the features), we observe the presence of six clusters. The first cluster contains all six MLP models, two RF models (RF40_E6D8 and RF40_E4D8), and one DT model (DT40_D12). The second cluster contains three ET models (ET40_E2D16, ET40_E6D8, and ET40_E4D8) and one model each of DT and RF (DT40_D4 and RF40_E2D8). The third cluster contains one ET model (ET40_E2D8). The fourth, fifth, and sixth clusters contain one, two, and three NB models, respectively.
- From Fig. 7.a (60 % of the features), we observe the presence of seven clusters. The first cluster contains one DT model (DT60_D12). The second cluster contains all six MLP models, three RF models (RF60_E4D8, RF60_E6D8, and RF60_E2D8), and one ET model (ET60_E2D16). The third cluster contains two ET models (ET60_E4D8 and ET60_E6D8) and one DT model (DT60_D4). The fourth cluster contains one ET model (ET60_E2D8). The fifth, sixth,

and seventh clusters contain one, one, and four NB models, respectively.

- From Fig. 7.b (100 % of the features), we observe the presence of three clusters. The first cluster contains all three RF models, all three ET models, both DT models, and four MLP models (MLP100_L46, MLP100_L56, MLP100_L16, and MLP100_L36). The second cluster contains one MLP model (MLP100_L26). The third cluster contains all six NB models and one MLP model (MLP100_L6).

TinyML models deployable on Arduino UNO were grouped using the SK test based on the Kappa metric, as shown in Figs. 4–7, to statistically identify the best-performing models. Among these, the best clusters were determined, and within each best cluster, the top model was selected using a voting system based on five evaluation criteria (MCC, Kappa, FM, SRAM, and latency). For the NF-ToN-IoT-v2 dataset, (1) the best clusters were identified for feature thresholds of 20 % and 40 %, as shown in Fig. 4, and 60 % and 100 %, as shown in Fig. 5; (2) applying the voting system to these best clusters produced the results presented in Table 7. Similarly, for the NF-BoT-IoT-v2 dataset, (1) the best clusters were determined for feature thresholds of 20 % and 40 %, as shown in Fig. 6, and 60 % and 100 %, as shown in Fig. 7; (2) the results obtained through the same voting process are reported in Table 8. In Tables 7 and 8, the models are ranked from best to least effective, with the top-performing models highlighted in bold for each threshold.

For NF-ToN-IoT-v2, as presented in Table 7, the analysis reveals that:

- With 13 features, DT20_D12 achieved the highest ranking, attaining an MCC of 0.849, a Kappa value of 0.846, a latency of 48 μ s, an SRAM usage of 22,934 bytes, and an FM size of 512 bytes, resulting in a BC score of 36.5.
- With 17 features, MLP40_L56 achieved the highest ranking, attaining an MCC of 0.868, a Kappa value of 0.867, a latency of 9.6 μ s, an SRAM usage of 6690 bytes, and an FM size of 806 bytes, resulting in a BC score of 24.
- With 20 features, MLP60_L56 achieved the highest ranking, attaining an MCC of 0.935, a Kappa value of 0.935, a latency of 9.6 μ s, an SRAM usage of 7010 bytes, and an FM size of 1126 bytes, resulting in a BC score of 24.
- With 33 features, RF100_E4D8 achieved the highest ranking, attaining an MCC of 0.913, a Kappa value of 0.913, a latency of 48 μ s, an SRAM usage of 192 bytes, and an FM size of 1452 bytes, resulting in a BC score of 6.

For NF-BoT-IoT-v2, as presented in Table 8, the analysis reveals that:

Table 7
BC ranking of TinyML models in the best cluster for the NF-ToN-IoT-v2 dataset and the three feature thresholds.

Algo	#Features	MCC	Kappa	Latency	SRAM	FM	Borda	Rank
DT20_D12	13	0.849	0.846	48	22,934	512	36.5	1
MLP20_L56		0.819	0.816	9.6	6410	526	33	2
MLP20_L46		0.813	0.810	9.6	6410	526	30.5	3
MLP20_L36		0.799	0.795	9.6	6410	526	28	4
RF20_E6D8		0.799	0.795	219.2	19,164	512	8.5	5
RF20_E4D8		0.794	0.790	171.2	15,176	512	6	6
MLP40_L56	17	0.868	0.867	9.6	6690	806	24	1
MLP40_L46		0.861	0.860	9.6	6690	806	21.5	2
MLP40_L36		0.854	0.853	9.6	6690	806	19	3
MLP40_L26		0.848	0.847	9.6	6690	806	16.5	4
MLP40_L16		0.840	0.838	9.6	6690	806	14	5
MLP60_L56	20	0.935	0.935	9.6	7010	1126	24	1
MLP60_L46		0.934	0.934	9.6	7010	1126	21.5	2
MLP60_L36		0.930	0.930	9.6	7010	1126	19	3
MLP60_L26		0.926	0.926	9.6	7010	1126	16.5	4
MLP60_L16		0.920	0.919	9.6	7010	1126	14	5
RF100_E4D8	33	0.913	0.913	192	28,406	1452	6	1
RF100_E2D8		0.910	0.910	124.8	16,002	1452	3.5	2

Table 8

BC ranking of TinyML models in the best cluster for the NF-BoT-IoT-v2 dataset and the three feature thresholds.

Algo	#Features	MCC	Kappa	Latency	SRAM	FM	Borda	Rank
ET20_E2D16	13	0.974	0.974	8	4730	353	88.5	1
DT20_D12		0.975	0.975	50.4	13,896	432	63.5	2
MLP20_L16		0.973	0.973	8.8	6330	446	50.5	3
MLP20_L46		0.973	0.973	8.8	6330	446	50	4
MLP20_L56		0.973	0.973	9.6	6330	446	47.5	5
XB20_E2D8		0.973	0.973	332	31,162	432	47	6
MLP20_L36		0.973	0.973	9.6	6330	446	47	6
MLP20_L26		0.973	0.973	9.6	6330	446	41.5	7
MLP20_L6		0.972	0.972	9.6	6330	446	41	8
DT20_D4		0.971	0.971	24	3694	432	35	9
RF20_E4D8		0.971	0.971	145.6	12,948	432	11	10
DT40_D12	17	0.977	0.977	50.4	16,480	652	53.5	1
RF40_E6D8		0.975	0.975	201.6	16,556	652	41	2
MLP40_L46		0.975	0.975	9.6	6550	666	40.5	3
MLP40_L36		0.974	0.974	9.6	6550	666	40	4
MLP40_L56		0.974	0.974	9.6	6550	666	39.5	5
MLP40_L26		0.974	0.974	9.6	6550	666	39	6
MLP40_L16		0.974	0.974	9.6	6550	666	38.5	7
MLP40_L6		0.973	0.973	9.6	6550	666	33	8
RF40_E4D8		0.973	0.973	144	12,096	652	17	9
DT60_D12	20	0.979	0.979	50.4	23,044	872		1
ET100_E2D16	33	0.978	0.978	8	5100	723	69	1
DT100_D12		0.987	0.987	33.6	23,416	1172	68	2
DT100_D4		0.979	0.979	17.6	4444	1172	67.5	3
ET100_E6D8		0.969	0.969	8	5100	723	63.5	4
RF100_E4D8		0.982	0.982	159.2	16,032	1172	59	5
ET100_E4D8		0.956	0.955	8	5100	723	58	6
RF100_E2D8		0.981	0.981	98.4	9760	1172	54.5	7
RF100_E6D8		0.982	0.982	220	21,858	1172	51.5	8
MLP100_L46		0.970	0.970	9.6	7070	1186	41	9
MLP100_L56		0.956	0.956	9.6	7070	1186	35.5	10
MLP100_L16		0.942	0.941	9.6	7070	1186	30	11
MLP100_L36		0.940	0.938	9.6	7070	1186	29.5	12

- With 13 features, ET20_E2D16 achieved the highest ranking, attaining an MCC of 0.974, a Kappa value of 0.974, a latency of 8 μ s, an SRAM usage of 4730 bytes, and an FM size of 353 bytes, resulting in a BC score of 88.5.
- With 17 features, DT40_D12 achieved the highest ranking, attaining an MCC of 0.977, a Kappa value of 0.977, a latency of 50.4 μ s, an SRAM usage of 16,480 bytes, and an FM size of 652 bytes, resulting in a BC score of 53.5.
- With 20 features, DT60_D12 ranked highest as the only model in the top cluster, achieving an MCC of 0.979, a Kappa value of 0.979, a latency of 50.4 μ s, an SRAM usage of 23,044 bytes, and an FM size of 872 bytes.
- With 33 features, ET100_E2D16 achieved the highest ranking, attaining an MCC of 0.978, a Kappa value of 0.978, a latency of 8 μ s, an SRAM usage of 5100 bytes, and an FM size of 723 bytes, resulting in a BC score of 69.

Moreover, to identify the top-performing TinyML model regardless of the datasets and feature thresholds, we also counted the number of occurrences of each TinyML technique in the best SK clusters and their BC ranks, as presented in Table 9. Only TinyML models belonging to the highest cluster were considered. The selection is determined by the number of times ML technique is ranked first in the top clusters. If multiple options have the same count, the total number of occurrences is considered. Consequently, (1) DT emerged as the top-performing TinyML technique, securing the first position three times, the second position once, and the third position once; (2) MLP followed, ranked first twice; (3) RF ranked first once with ten occurrences in the best cluster; (4) ET ranked first once with four occurrences in the best cluster; (5) XGB had one occurrence in the best cluster; and (5) in the last position, NB had zero occurrences in the best cluster.

In summary, the primary results of the evaluation indicated that singular TinyML models performed better than ensemble models for

Table 9

Appearance of the TinyML classifiers in the first SK cluster regardless of the NF-ToN-IoT-v2 and NF-BoT-IoT-v2 datasets, and the feature thresholds.

Model	Occurrences in the best cluster	BC ranking
DT	7	First 3 times Second 1 time Third 1 time Fourth 1 time Tenth 1 time
ET	4	First 1 time Second 1 time Sixth 1 time Eighth 1 time Sixth 1 time
XB	1	Sixth 1 time
MLP	29	First 2 times Second 3 times Third 5 times Fourth 5 times Fifth 4 times Sixth 1 time Seventh 2 times Eighth 2 times Tenth 1 time Eleventh 1 time Twelfth 1 time Thirteenth 1 time
RF	10	First 1 time Second 2 times Fifth 1 time Sixth 2 times Eighth 1 time Ninth 2 times Eleventh 1 time
NB	0	-

multiclass classification in the IDS-IoT context. Notably, the ensemble models in this study employed a limited number of estimators, which may have impacted their performance. Although ensemble models are typically more stable across datasets, well-optimized singular models can deliver superior results in specific contexts. This highlights the importance of selecting models based on the specific requirements and constraints of the application. Furthermore, the comparison of top-performing models across different feature thresholds (13, 17, 20, and 33 features) shows that their performance is relatively consistent for the NF-BoT-IoT-v2 dataset, in contrast to the NF-ToN-IoT-v2 dataset, where performance remains consistent from 20 features upwards, as detailed in [Tables 7 and 8](#). Specifically, the best model: (1) for NF-ToN-IoT-v2 is MLP60_L56, achieving a Kappa of 0.935, MCC of 0.935, latency of 9.6 μ s, SRAM of 7010 bytes, and FM of 1126 bytes using only 20 features; and (2) for NF-BoT-IoT-v2 is ET20_E2D16, with a Kappa of 0.974, MCC of 0.974, latency of 8 μ s, SRAM of 4730 bytes, and FM of 353 bytes using 13 features.

5.4. Performance results comparison

This section compares the empirical results of our study with those from previous research. [Table 10](#) presents the performance metrics of the top performing models in our study alongside results from earlier studies. The comparison focuses on three main aspects: performance metrics; hardware efficiency (including memory usage and latency); and the number of features selected from the dataset. [Table 10](#) serves as a reference point for evaluating the advancements and contributions of the proposed method.

The analysis highlights a key achievement: our proposed TinyML method consistently outperforms existing approaches in terms of SRAM, Flash memory usage, and latency, demonstrating its effectiveness as a resource-efficient IDS tool. Furthermore, unlike other studies that primarily rely on accuracy (a metric sensitive to imbalanced data) our evaluation incorporates more robust measures (MCC and Kappa), showcasing superior performance and feature reduction. The findings summarized in [Table 10](#) underscore the progress achieved by our method, paving the way for the development of more efficient and adaptable IDS solutions that address the limitations of ensemble and single techniques approaches.

6. Threats of validity

Ensuring the trustworthiness of this study requires clearly defining the scope of valid conclusions. Due to the large size of several datasets and the extensive time required for model parameter tuning, processing them for multiclass classification tasks was impractical. Additionally, given the critical importance of cybersecurity, relying solely on a single feature set is insufficient for evaluating the applicability or reliability of features.

We chose FS based on prior studies of NetFlow data in IoT intrusion detection. Including additional methods like principal component analysis, Lasso regression, or recursive feature elimination would have increased computational complexity and processing time, especially for

large datasets. Testing on Arduino also requires significant time. Therefore, we focused on efficiency while ensuring robust FS and plan to explore other techniques in future work.

Model construction typically revolves around the hyperparameter ranges used in this study. However, utilizing different ranges for these hyperparameters can also influence the performance of the models, potentially making them more suitable for deployment in TinyML environments. In addition, Using the Arduino UNO, a highly constrained microcontroller with minimal memory, allows for the generalization of findings to other embedded devices in the IoT field. However, deploying TinyML models with certain parameters may result in large models that are unsupported by the Arduino UNO due to memory limitations. To deploy and evaluate TinyML models that could not be implemented in this study, it is advantageous to utilize a range of embedded devices with more powerful microcontrollers or CPUs, facilitating the deployment of different TinyML models across various embedded device environments.

The outcomes of this analysis provide valuable insights for future research and discussions, particularly in the development of more reliable and practical TinyML models for multiclass classification in IoT context. Furthermore, periodic model retraining is essential for addressing evolving cyber threats, as static models may lose effectiveness over time. Although TinyML models are designed to operate independently without continuous updates to guard against adversarial attacks like data poisoning, retraining through centralized MLOps frameworks or federated learning can enhance adaptability and improve effectiveness against emerging threats. Future efforts will include addressing adversarial attacks, such as evasion techniques, and incorporating ensemble DL models to improve the robustness and applicability of IDS based TinyML on IoT environments.

7. Conclusion and future work

This study examined and compared the performance of ensemble and singular TinyML classifiers using two NetFlow v9 IoT datasets deployed on Arduino UNO devices, a resource-constrained device with minimal memory, used to extend the findings to other IoT embedded devices. The research focused on singular techniques (DT, MLP, and NB) and ensemble techniques (ET, XGB, and RF), utilizing DT as base learners for multiclass classification tasks. To enhance model performance and robustness, two FS techniques (ANOVA and mRMR) were employed, along with four different thresholds (20 %, 40 %, 60 %, and 100 %). The evaluation and ranking of these models were conducted using the SK test based on Kappa and the BC method based on two prediction metrics (MCC and Kappa) and three TinyML metrics (latency, SRAM, and FM). The key findings of this research are summarized as follows:

- **(RQ1):** For singular techniques, increasing the number of features led to higher usage of SRAM and FM. For MLP and NB, all generated models are supported by the Arduino UNO, and changing their hyperparameters does not affect SRAM, FM, or latency. In contrast, not all DT models are supported; only those with trees depths of 4

Table 10
performance results comparison with previous studies.

Authors and year of publication	Dataset	Device	# of features	Best performance					
				MCC	Kappa	SRAM	FM	Latency	Accuracy
Our study	NF-ToN-IoT-v2	Arduino UNO	13	0.974	0.974	4730 B	353 B	8 μ s	-
	NF-BoT-IoT-v2		17	0.935	0.935	7010 B	1126 B	9.6 μ s	-
Hussain et al. [36]	Private dataset	Raspberry Pi	All features	-	-			-	86.33 %
Dehrouyeh et al. [10]	CICIDS2017	ESP32				8.6953 KB	0.9023 KB	246.825 μ s	80.83 %
Al-Waisi et al. [37]	Private dataset	• Raspberry Pi • Arduino Nano						24.6 s	96.9 %
Fusco et al. [61]	TON IoT	• Arduino Portenta H7 • ESP32				114.492 KB		362 ms	84.03 %

and 12 are supported by the Arduino UNO. Increasing the tree depth for DT models results in higher usage of SRAM and FM.

- **(RQ2):** For the ensemble ML techniques used, increasing the number of features increased the usage of SRAM and FM. For ET, the supported models have the same latency for all thresholds, and increasing the number of estimators and tree depths does not increase the usage of SRAM and FM. For RF, increasing the number of estimators and tree depth does not affect FM but does increase SRAM usage. For XGB, only the XGB20_E2D8 model is deployed over NF-BoT-IoT-v2 using a small number of estimators and a small tree depth (E2D8).
- **(RQ3):** Singular TinyML models outperformed ensemble models for multiclass classification in the IDS-IoT context. Notably, the ensemble models in this study used a small number of estimators, which may have influenced their performance. While ensemble models are generally more stable across datasets, optimized singular models can achieve superior performance in specific contexts, emphasizing the need to align model selection with application requirements. The best models identified are: (1) MLP60_L56 for NF-ToN-IoT-v2 using 20 features, and (2) ET20_E2D16 for NF-BoT-IoT-v2 using 13 features.

Ongoing work intends to investigate classical computer networks and conduct real-world tests in IoT environments, in addition to incorporating energy consumption as an additional metric and exploring the trade-off between accuracy, memory usage, latency, and energy efficiency to optimize model performance for resource-constrained IoT devices. This will provide further validation of our model's robustness and adaptability in detecting intrusions under real-world conditions. Moreover, other TinyML techniques and embedded devices will be investigated for attack detection in an IoT environment in order to confirm or refute the findings of the present study. Additionally, developing real-time IDS will be an important step toward enhancing the practical applicability of our model in real-world IoT networks. Furthermore, we intend to incorporate techniques to address the class imbalance issue, including oversampling, undersampling, and cost-sensitive algorithms. We also aim to explore additional feature reduction techniques to assess their impact on model performance and computational efficiency in resource-constrained environments. Additionally, we will investigate adversarial training and anomaly detection to enhance the TinyML model's resilience against evasion attacks in IoT environments. Additionally, using other embedded devices with different MCUs and CPUs will allow for the deployment of additional

models with various hyperparameters, expanding the scope of experimentation. Efforts will also focus on developing retraining mechanisms for TinyML-based IDS to adapt to evolving cyber threats while ensuring lightweight and resource-efficient deployments. This will include integrating MLOps frameworks to automate model retraining and deployment. Furthermore, federated learning will be explored to enable decentralized periodic updates, ensuring the security and adaptability of TinyML-based IDS in real-world IoT networks.

Ethical approval

This article does not contain any studies with human participants or animals performed by any of the authors.

Funding

Not applicable. No funding was received by the authors for conducting this research.

Availability of data and materials

This article uses two public datasets, which are publicly available online at: (https://staff.itee.uq.edu.au/marius/NIDS_datasets/).

The code used in this study is available on GitHub under an MIT open-source license, ensuring transparency and reproducibility. You can access the code at: (<https://github.com/AbdelErrahmane/MicroprocessorsAndMicrosystems>)

CRedit authorship contribution statement

Abderahmane Hamdouchi: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Software, Visualization, Writing – original draft. **Ali Idri:** Conceptualization, Formal analysis, Funding acquisition, Methodology, Project administration, Resources, Supervision, Validation, Visualization, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Performance of models using Arduino NANO metrics

Tables A1 and A2 present the performance criteria of Arduino UNO using NF-ToN-IoT-v2 for singular and ensemble models, respectively. Similarly, Tables A3 and A4 display the performance criteria of Arduino UNO using NF-BoT-IoT-v2 for singular and ensemble models. In Tables A1–A4, RTO represents the “region ‘text’ overflowed” compilation error, SAL represents the “size of array is too large” compilation error, and OMA represents the “out of memory allocating” compilation error.

Table A1

Performance of deployed and non-deployed TinyML singular models on Arduino Uno using the NF-ToN-IoT-v2 dataset.

DT (MicroMLGen)				MLP (EMLearn)				NB (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
DT20_D4	23.2	32,256	512	MLP20_L6	9.6	6410	526	NB20_S1e-04	32,445.6	8546	512
DT20_D12	48	22,934	512	MLP20_L16	9.6	6410	526	NB20_S1e-05	32,654.4	8546	512
DT20_D20		32,256	512	MLP20_L26	9.6	6410	526	NB20_S1e-06	33,359.2	8546	512
DT20_D28	RTO 8508 bytes			MLP20_L36	9.6	6410	526	NB20_S1e-07	32,537.6	8546	512
DT20_D36	RTO 10,300 bytes			MLP20_L46	9.6	6410	526	NB20_S1e-08	32,705.6	8546	512
DT20_D40	RTO 10,486 bytes			MLP20_L56	9.6	6410	526	NB20_S1e-09	33,190.4	8546	512
DT40_D4	23.2	32,256	792	MLP40_L6	9.6	6690	806	NB40_S1e-04	56,776.8	13,600	792
DT40_D12		43,438	792	MLP40_L16	9.6	6690	806	NB40_S1e-05	56,982.4	13,600	792
DT40_D20	RTO 173,642 bytes			MLP40_L26	9.6	6690	806	NB40_S1e-06	57,830.4	13,600	792

(continued on next page)

Table A1 (continued)

DT (MicroMLGen)				MLP (EMLearn)				NB (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
DT40_D28	RTO 360,270 bytes			MLP40_L36	9.6	6690	806	NB40_S1e-07	56,798.4	13,600	792
DT40_D36	RTO 391,534 bytes			MLP40_L46	9.6	6690	806	NB40_S1e-08	57,035.2	13,600	792
DT40_D40	RTO 392,272 bytes			MLP40_L56	9.6	6690	806	NB40_S1e-09	57,605.6	13,600	792
DT60_D4	22.4	4558	1112	MLP60_L6	9.6	7010	1126	NB60_S1e-04	91,724.8	20,180	1112
DT60_D12		52,056	1112	MLP60_L16	9.6	7010	1126	NB60_S1e-05	92,131.2	20,180	1112
DT60_D20	RTO 382,092 bytes			MLP60_L26	9.6	7010	1126	NB60_S1e-06	93,553.6	20,180	1112
DT60_D28	RTO 1039,988 bytes			MLP60_L36	9.6	7010	1126	NB60_S1e-07	92,084.8	20,180	1112
DT60_D36	RTO 1395,286 bytes			MLP60_L46	9.6	7010	1126	NB60_S1e-08	92,372.8	20,180	1112
DT60_D40	RTO 1481,536 bytes			MLP60_L56	9.6	7010	1126	NB60_S1e-09	93,665.6	20,180	1112
DT100_D4	23.2	4854	1452	MLP100_L6	9.6	7350	1466	NB100_S1e-04	70,678.4	14,248	1452
DT100_D12		61,330	1452	MLP100_L16	9.6	7350	1466	NB100_S1e-05	66,005.6	14,248	1452
DT100_D20	RTO 536,552 bytes			MLP100_L26	9.6	7350	1466	NB100_S1e-06	68,681.6	14,248	1452
DT100_D28	RTO 1396,588 bytes			MLP100_L36	9.6	7350	1466	NB100_S1e-07	71,159.2	14,280	1452
DT100_D36	RTO 2275,410 bytes			MLP100_L46	9.6	7350	1466	NB100_S1e-08	68,646.4	14,280	1452
DT100_D40	RTO 2789,108 bytes			MLP100_L56	9.6	7350	1466	NB100_S1e-09	67,804	14,408	1452

Table A2

Performance of deployed and non-deployed TinyML singular models on Arduino Uno using the NF-BoT-IoT-v2 dataset.

DT (MicroMLGen)				MLP (EMLearn)				NB (MicroMLGen)s			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
DT20_D4	24	3694	432	MLP20_L6	9.6	6330	446	NB20_S1e-04	12,723.2	6488	432
DT20_D12	50.4	13,896	432	MLP20_L16	8.8	6330	446	NB20_S1e-05	12,771.2	6488	432
DT20_D20	-	42,440	432	MLP20_L26	9.6	6330	446	NB20_S1e-06	12,812	6488	432
DT20_D28	-	71,696	432	MLP20_L36	9.6	6330	446	NB20_S1e-07	12,719.2	6488	432
DT20_D36	-	82,536	432	MLP20_L46	8.8	6330	446	NB20_S1e-08	12,732	6488	432
DT20_D40	-	86,674	432	MLP20_L56	9.6	6330	446	NB20_S1e-09	12,804.8	6488	432
DT40_D4	24	3918	652	MLP40_L6	9.6	6550	666	NB40_S1e-04	21,900.8	8472	652
DT40_D12	50.4	16,480	652	MLP40_L16	9.6	6550	666	NB40_S1e-05	21,964	8472	652
DT40_D20	-	77,672	652	MLP40_L26	9.6	6550	666	NB40_S1e-06	22,090.4	8472	652
DT40_D28	RTO 13,564 bytes			MLP40_L36	9.6	6550	666	NB40_S1e-07	21,886.4	8472	652
DT40_D36	RTO 40,096 bytes			MLP40_L46	9.6	6550	666	NB40_S1e-08	21,904.8	8472	652
DT40_D40	RTO 46,730 bytes			MLP40_L56	9.6	6550	666	NB40_S1e-09	22,048.8	8472	652
DT60_D4	24	4146	872	MLP60_L6	9.6	6768	886	NB60_S1e-04	31,448.8	10,694	872
DT60_D12	50.4	23,044	872	MLP60_L16	9.6	6768	886	NB60_S1e-05	31,514.4	10,694	872
DT60_D20	-	117,306	872	MLP60_L26	9.6	6768	886	NB60_S1e-06	31,737.6	10,694	872
DT60_D28	RTO 84,150 bytes			MLP60_L36	9.6	6768	886	NB60_S1e-07	31,476.8	10,694	872
DT60_D36	RTO 112,036 bytes			MLP60_L46	9.6	6768	886	NB60_S1e-08	31,521.6	10,694	872
DT60_D40	RTO 119,268 bytes			MLP60_L56	9.6	6768	886	NB60_S1e-09	31,732	10,694	872
DT100_D4	17.6	4444	1172	MLP100_L6	9.6	7070	1186	NB100_S1e-04	31,736	9188	1172
DT100_D12	33.6	23,416	1172	MLP100_L16	9.6	7070	1186	NB100_S1e-05	29,991.2	9188	1172
DT100_D20	-	99,860	1172	MLP100_L26	9.6	7070	1186	NB100_S1e-06	29,443.2	9188	1172
DT100_D28	RTO 81,274 bytes			MLP100_L36	9.6	7070	1186	NB100_S1e-07	32,092	9188	1172
DT100_D36	RTO 122,144 bytes			MLP100_L46	9.6	7070	1186	NB100_S1e-08	29,943.2	9208	1172
DT100_D40	RTO 154,122 bytes			MLP100_L56	9.6	7070	1186	NB100_S1e-09	30,190.4	9208	1172

Table A3

Performance of deployed and non-deployed TinyML ensemble models on Arduino Uno using the NF-ToN-IoT-v2 dataset.

ET (EMLearn)				RF (MicroMLGen)				XB (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
ET20_E2D8	8	4770	393	RF20_E2D8	23.2	9526	1536	XB20_E2D8	-	87,366	512
ET20_E2D16	8	4770	393	RF20_E2D16	-	82,868	1536	XB20_E2D16	RTO 221,098 bytes		
ET20_E4D8	8	4770	393	RF20_E4D8	171.2	15,176	512	XB20_E4D8	RTO 52,950 bytes		
ET20_E4D16	8	4770	393	RF20_E4D16	RTO 88,182 bytes			XB20_E4D16	RTO 630,932 bytes		
ET20_E6D8	8	4770	393	RF20_E6D8	219.2	19,164	512	XB20_E6D8	RTO 156,278 bytes		
ET20_E6D16	SAL			RF20_E6D16	RTO 228,174 bytes			XB20_E6D16	RTO 1073,502 bytes		
ET40_E2D8	8	4910	533	RF40_E2D8	128	12,834	792	XB40_E2D8	-	113,474	792
ET40_E2D16	8	4910	533	RF40_E2D16	RTO 153,026 bytes			XB40_E2D16	RTO 742,206 bytes		
ET40_E4D8	8	4910	533	RF40_E4D8	180.8	20,970	792	XB40_E4D8	RTO 109,104 bytes		
ET40_E4D16	SAL			RF40_E4D16	RTO 493,174 bytes			XB40_E4D16	RTO 1771,160 bytes		
ET40_E6D8	8	4910	533	RF40_E6D8	243.2	30,828	792	XB40_E6D8	RTO 235,302 bytes		
ET40_E6D16	SAL			RF40_E6D16	RTO 802,346 bytes			XB40_E6D16	RTO 2825,566 bytes		
ET60_E2D8	8	5070	693	RF60_E2D8	134.4	14,154	1112	XB60_E2D8	-	127,784	1112
ET60_E2D16	SAL			RF60_E2D16	RTO 202,750 bytes			XB60_E2D16	RTO 1125,346 bytes		
ET60_E4D8	8	5070	693	RF60_E4D8	200	25,288	1112	XB60_E4D8	RTO 131,266 bytes		

(continued on next page)

Table A3 (continued)

ET (EMLearn)				RF (MicroMLGen)				XB (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
ET60_E4D16	SAL			RF60_E4D16	RTO 565,854 bytes			XB60_E4D16	RTO 2493,724 bytes		
ET60_E6D8	8	5070	693	RF60_E6D8	-	35,768	1112	XB60_E6D8	RTO 266,728 bytes		
ET60_E6D16	SAL			RF60_E6D16	RTO 909,156 bytes			XB60_E6D16	RTO 3922,810 bytes		
ET100_E2D8	8	5240	863	RF100_E2D8	124.8	16,002	1452	XB100_E2D8	-	124,706	1452
ET100_E2D16	8	5240	863	RF100_E2D16	RTO 372,298 bytes			XB100_E2D16	RTO 1421,432 bytes		
ET100_E4D8	8	5240	863	RF100_E4D8	192	28,406	1452	XB100_E4D8	RTO 133,106 bytes		
ET100_E4D16	SAL			RF100_E4D16	RTO 937,294 bytes			XB100_E4D16	RTO 3130,688 bytes		
ET100_E6D8	8	5240	863	RF100_E6D8	-	40,454	1452	XB100_E6D8	RTO 277,078 bytes		
ET100_E6D16	SAL			RF100_E6D16	RTO 1454,064 bytes			XB100_E6D16	OMA 10,480 bytes		

Table A4

Performance of deployed and non-deployed TinyML ensemble models on Arduino Uno using the NF-BoT-IoT-v2 dataset.

ET (EMLearn)				RF (MicroMLGen)				XGB (MicroMLGen)			
Model	Latency	SRAM	FM	Model	Latency	SRAM	FM	Model	Latency	SRAM	FM
ET20_E2D8	8	4730	353	RF20_E2D8	95.2	7808	432	XGB20_E2D8	332	31,162	432
ET20_E2D16	8	4730	353	RF20_E2D16	-	39,162	432	XGB20_E2D16	-	119,438	432
ET20_E4D8	8	4730	353	RF20_E4D8	145.6	12,948	432	XGB20_E4D8	-	62,396	432
ET20_E4D16	SAL			RF20_E4D16	-	95,146	432	XGB20_E4D16	RTO 140,040 bytes		
ET20_E6D8	8	4730	353	RF20_E6D8	197.6	17,440	432	XGB20_E6D8	-	96,450	32,256
ET20_E6D16	SAL			RF20_E6D16	RTO 43,836 bytes			XGB20_E6D16	RTO 303,880 bytes		
ET40_E2D8	8	4840	463	RF40_E2D8	92	7618	652	XGB40_E2D8	-	40,638	652
ET40_E2D16	8	4840	463	RF40_E2D16	-	52,374	652	XGB40_E2D16	RTO 147,194 bytes		
ET40_E4D8	8	4840	463	RF40_E4D8	144	12,096	652	XGB40_E4D8	-	85,002	652
ET40_E4D16	SAL			RF40_E4D16	RTO 30,118 bytes			XGB40_E4D16	RTO 464,422 bytes		
ET40_E6D8	8	4840	463	RF40_E6D8	201.6	16,556	652	XGB40_E6D8	RTO 740 bytes		
ET40_E6D16	SAL			RF40_E6D16	RTO 104,520 bytes			XGB40_E6D16	RTO 780,852 bytes		
ET60_E2D8	8	4950	573	RF60_E2D8	97.6	9390	872	XGB60_E2D8	-	45,898	872
ET60_E2D16	8	4950	573	RF60_E2D16	-	90,752	872	XGB60_E2D16	RTO 255,608 bytes		
ET60_E4D8	8	4950	573	RF60_E4D8	156	14,426	872	XGB60_E4D8	-	101,414	872
ET60_E4D16	SAL			RF60_E4D16	RTO 105,170 bytes			XGB60_E4D16	RTO 678,592 bytes		
ET60_E6D8	8	4950	573	RF60_E6D8	215.2	20,380	872	XGB60_E6D8	RTO 25,134 bytes		
ET60_E6D16	SAL			RF60_E6D16	RTO 265,496 bytes			XGB60_E6D16	RTO 1086,592 bytes		
ET100_E2D8				RF100_E2D8	98.4	9760	1172	XGB100_E2D8	-	40,654	1172
ET100_E2D16	8	5100	723	RF100_E2D16	-	88,840	1172	XGB100_E2D16	RTO 260,160 bytes		
ET100_E4D8	8	5100	723	RF100_E4D8	159.2	16,032	1172	XGB100_E4D8	-	97,158	1172
ET100_E4D16	SAL			RF100_E4D16	RTO 108,568 bytes			XGB100_E4D16	RTO 749,292 bytes		
ET100_E6D8	8	5100	723	RF100_E6D8	220	21,858	1172	XGB100_E6D8	RTO 25,894 bytes		
ET100_E6D16	SAL			RF100_E6D16	RTO 260,620 bytes			XGB100_E6D16	RTO 1307,000 bytes		

Data availability

This study uses two public datasets that are publicly available online at: https://staff.itee.uq.edu.au/marius/NIDS_datasets/

References

- [1] R. Sanchez-Iborra, A. Zoubir, A. Hamdouchi, A. Idri, A. Skarmeta, Intelligent and efficient IoT through the cooperation of TinyML and edge computing, *Informatica* 0 (2023) 1–22, <https://doi.org/10.15388/22-INFOR505>.
- [2] Y. Lu, L.D. Xu, Internet of things (IoT) cybersecurity research: a review of current research topics, *IEEE Internet Things J.* 6 (2019) 2103–2115, <https://doi.org/10.1109/JIOT.2018.2869847>.
- [3] H.J. Liao, C.H. Richard Lin, Y.C. Lin, K.Y. Tung, Intrusion detection system: a comprehensive review, *J. Netw. Comput. Appl.* 36 (2013) 16–24, <https://doi.org/10.1016/J.JNCA.2012.09.004>.
- [4] M.D. Kunal, Machine learning approach to IDS: a comprehensive review, in: *Proceedings of the 3rd International Conference on Electronics and Communication and Aerospace Technology, ICECA 2019, 2019*, pp. 117–121, <https://doi.org/10.1109/MCAS.2020.3005467>.
- [5] R. Sanchez-Iborra, A.F. Skarmeta, TinyML-enabled frugal smart objects: challenges and opportunities, *IEEE Circuits Syst. Mag.* 20 (2020) 4–18, <https://doi.org/10.1109/MCAS.2020.3005467>.
- [6] A. Mohammed, R. Kora, A comprehensive review on ensemble deep learning: opportunities and challenges, *J. King Saud Univ. - Comput. Inf. Sci.* 35 (2023) 757–774, <https://doi.org/10.1016/J.JKSUCI.2023.01.014>.
- [7] S. Sicari, A. Rizzardi, L.A. Grieco, A. Coen-Porisini, Testing and evaluating a security-aware pub and sub protocol in a fog-driven IoT environment, *Lect. Notes Comput. Sci. (Incl. Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinform.)* 12338 LNCS (2020) 183–197, https://doi.org/10.1007/978-3-030-61746-2_14.
- [8] H. Attou, A. Guezaz, S. Benkirane, M. Azrou, Y. Farhaoui, Cloud-based intrusion detection approach using machine learning techniques, *Big Data Min. Anal.* 6 (2023) 311–320, <https://doi.org/10.26599/BDMA.2022.9020038>.
- [9] N. Tekin, A. Acar, A. Aris, A.S. Uluagac, V.C. Gungor, Energy consumption of on-device machine learning models for IoT intrusion detection, *Internet Things* 21 (2023) 100670, <https://doi.org/10.1016/J.IOT.2022.100670>.
- [10] F. Dehrouyeh, L. Yang, F.B. Ajaei, A. Shami, On TinyML and cybersecurity: electric vehicle charging infrastructure use case, (2024). <https://arxiv.org/abs/2404.16894v2> (accessed May 14, 2024).
- [11] H. Peng, F. Long, C. Ding, Feature selection based on mutual information: criteria of max-dependency, max-relevance, and min-redundancy, *IEEE Trans. Pattern Anal. Mach. Intell.* 27 (2005) 1226–1238, <https://doi.org/10.1109/TPAMI.2005.159>.
- [12] K.J. Johnson, R.E. Synovec, Pattern recognition of jet fuels: comprehensive GC×GC with ANOVA-based feature selection and principal component analysis, *Chemom. Intell. Lab. Syst.* 60 (2002) 225–237, [https://doi.org/10.1016/S0169-7439\(01\)00198-8](https://doi.org/10.1016/S0169-7439(01)00198-8).
- [13] G.M. Cramer, R.A. Ford, R.L. Hall, Estimation of toxic hazard—a decision tree approach, *Food Cosmet. Toxicol.* 16 (1976) 255–276, [https://doi.org/10.1016/S0015-6264\(76\)80522-6](https://doi.org/10.1016/S0015-6264(76)80522-6).
- [14] A.H. Jahromi, M. Taheri, A non-parametric mixture of Gaussian naive Bayes classifiers based on local independent features, in: *19th CSI International Symposium on Artificial Intelligence and Signal Processing, AISP 2017 2018-January, 2017*, pp. 209–212, <https://doi.org/10.1109/AISP.2017.8324083>.
- [15] M. Popescu, V. Balas, L.P.-P.-on C. and, undefined, Multilayer perceptron and neural networks, *Acad.Edu* (2009) n.d. <https://www.academia.edu/download/69679997/29-485.pdf>. accessed May 13, 2024

- [16] A. Cutler, D.R. Cutler, J.R. Stevens, Random forests, ensemble machine learning (2012) 157–175, https://doi.org/10.1007/978-1-4419-9326-7_5.
- [17] T. Chen, C. Guestrin, XGBoost: a scalable tree boosting system, in: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining 13-17-August-2016, 2016, pp. 785–794, <https://doi.org/10.1145/2939672.2939785>.
- [18] P. Geurts, D. Ernst, L. Wehenkel, Extremely randomized trees, Mach. Learn. 63 (2006) 3–42, <https://doi.org/10.1007/S10994-006-6226-1/METRICS>.
- [19] M. Sarhan, S. Layeghy, M. Portmann, Towards a standard feature set for network intrusion detection system datasets, Mob. Netw. Appl. 27 (2022) 357–370, <https://doi.org/10.1007/S11036-021-01843-0/FIGURES/4>.
- [20] T.Z. Phyu, N.N. Oo, Performance comparison of feature selection methods, MATEC Web Conf. 42 (2016) 06002, <https://doi.org/10.1051/MATECONF/20164206002>.
- [21] H. Zouhri, A. Idri, A. Ratnani, Evaluating the impact of filter-based feature selection in intrusion detection systems, Int. J. Inf. Secur. (2023) 1–27, <https://doi.org/10.1007/S10207-023-00767-Y/TABLES/17>.
- [22] A. Hamdouchi, A. Idri, Enhancing IoT security through boosting and feature reduction techniques for multiclass intrusion detection, Neural Comput. Appl. 2025 (2025) 1–24, <https://doi.org/10.1007/S00521-025-11001-2>.
- [23] M. Nakashima, Y. Kim, J. Kim, J. Kim, A. Sim, Automated feature selection for anomaly detection in, Netw. Traffic Data 1 (2018) 27, <https://doi.org/10.1145/1122445.1122456>.
- [24] S.S. Dhaliwal, A. Al Nahid, R. Abbas, Effective intrusion detection system using XGBoost, Information 9 (2018) 149, <https://doi.org/10.3390/INFO9070149>, 2018Vol., Page 149 9.
- [25] D.S. Kurniawati, M.Y. Darmawijoyo, I. Bin Bin, A.M. Bamhdi, R. Budiarto, CICIDS-2017 dataset feature analysis with information gain for anomaly detection, IEEE Access 8 (2020) 132911–132921, <https://doi.org/10.1109/ACCESS.2020.3009843>.
- [26] M. Jans, P. Soffer, T. Jock, A coefficient of agreement for nominal scales, Educ. Psychol. Meas. 20 (1960) 37–46, <https://doi.org/10.1177/001316446002000104>.
- [27] P. Baldi, S. Brunak, Y. Chauvin, C.A.F. Andersen, H. Nielsen, Assessing the accuracy of prediction algorithms for classification: an overview, Bioinformatics 16 (2000) 412–424, <https://doi.org/10.1093/BIOINFORMATICS/16.5.412>.
- [28] U. Brinkschulte, M. Pacher, Improving the real-time behaviour of a multithreaded java microcontroller by control theory and model based latency prediction, in: Proceedings - International Workshop on Object-Oriented Real-Time Dependable Systems, WORDS, 2005, pp. 82–93, <https://doi.org/10.1109/WORDS.2005.38>.
- [29] J. Pallister, K. Eder, S.J. Hollis, Optimizing the flash-RAM energy trade-off in deeply embedded systems, in: Proceedings of the 2015 IEEE/ACM International Symposium on Code Generation and Optimization 2015, CGO, 2015, pp. 115–124, <https://doi.org/10.1109/CGO.2015.7054192>.
- [30] J.B. Shaik, S. Singhal, N. Goel, Analysis of SRAM metrics for data dependent BTI degradation and process variability, Integration 72 (2020) 148–162, <https://doi.org/10.1016/J.VLSI.2020.01.006>.
- [31] A.J. Scott, M. Knott, A cluster analysis method for grouping means in the analysis of variance, Biometrics 30 (1974) 507, <https://doi.org/10.2307/2529204>.
- [32] D.G. Saari, Decisions and elections, Decis. Elect. (2001), <https://doi.org/10.1017/CBO9780511606076>.
- [33] I. Fedorov, M. Stamenovic, C. Jensen, L.C. Yang, A. Mandell, Y. Gan, M. Mattina, P. N. Whatmough, TinyLSTMs: efficient neural speech enhancement for hearing aids, in: Proceedings of the Annual Conference of the International Speech Communication Association, INTERSPEECH 2020-October, 2020, pp. 4054–4058, <https://doi.org/10.21437/Interspeech.2020-1864>.
- [34] N. Schilling, M. Wistuba, L. Drumond, L. Schmidt-Thieme, Hyperparameter optimization with factorized multilayer perceptrons, Lect. Notes Comput. Sci. (Incl. Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinform.) 9285 (2015) 87–103, https://doi.org/10.1007/978-3-319-23525-7_6.
- [35] J.C. Lévesque, C. Gagné, R. Sabourin, Bayesian hyperparameter optimization for ensemble learning, 32nd Conference on Uncertainty in Artificial Intelligence 2016, UAI 2016, <https://arxiv.org/abs/1605.06394v1>, 2016 accessed May 13, 2024.
- [36] A. Hussain, N. Abughanam, J. Qadir, A. Mohamed, Jamming detection in IoT wireless networks: an edge-AI based approach, ACM Int. Conf. Proceeding Ser. (2022) 57–64, <https://doi.org/10.1145/3567445.3567456>.
- [37] Z. Al-Waisi, S. Soderi, T. Kumar, E. Harjula, Securing constrained IoT systems: a lightweight machine learning approach for anomaly detection and prevention, (2024), <https://doi.org/10.2139/SSRN.4821986>.
- [38] I. Katib, E. Albassam, S.A. Sharaf, M. Ragab, Safeguarding IoT consumer devices: deep learning with TinyML driven real-time anomaly detection for predictive maintenance, Ain Shams Eng. J. 16 (2025) 103281, <https://doi.org/10.1016/J.ASEJ.2025.103281>.
- [39] Z. Alwaisi, S. Soderi, Towards robust IoT defense: comparative statistics of attack detection in resource-constrained scenarios, in: Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering, LNICT 524, LNICT, 2024, pp. 272–291, https://doi.org/10.1007/978-3-031-72524-1_20.
- [40] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A.N. Anwar, TON-IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems, IEEE Access 8 (2020) 165130–165150, <https://doi.org/10.1109/ACCESS.2020.3022862>.
- [41] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: bot-IoT dataset, Future Gener. Comput. Syst. 100 (2019) 779–796, <https://doi.org/10.1016/J.FUTURE.2019.05.041>.
- [42] G. Naidu, T. Zuva, E.M. Sibanda, A review of evaluation metrics in machine learning algorithms, in: Lecture Notes in Networks and Systems 724 LNNS, 2023, pp. 15–25, https://doi.org/10.1007/978-3-031-35314-7_2/FIGURES/3.
- [43] H. He, Y. Ma, Imbalanced learning: foundations, algorithms, and applications, Imbalanced learning: foundations, Algorithms Appl. (2013) 1–210, <https://doi.org/10.1002/9781118646106>.
- [44] M. Awad, S. Fraihat, K. Salameh, A. Al Redhaei, Examining the suitability of NetFlow features in detecting IoT network intrusions, Sensors 22 (2022) 6164, <https://doi.org/10.3390/S22166164>, 2022, Vol. 22, Page 6164.
- [45] M.H. Khan, A.R. Javed, Z. Iqbal, M. Asim, A.I. Awad, DivaCAN: detecting in-vehicle intrusion attacks on a controller area network using ensemble learning, Comput. Secur. 139 (2024) 103712, <https://doi.org/10.1016/J.COSE.2024.103712>.
- [46] Q.R.S. Fitni, K. Ramli, Implementation of ensemble learning and feature selection for performance improvements in anomaly-based intrusion detection systems, in: Proceedings - 2020 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology 2020, IAICT, 2020, pp. 118–124, <https://doi.org/10.1109/IAICT50021.2020.9172014>.
- [47] M.A. Jabbar, R. Aluvalu, S. Sai Satyanarayana Reddy, Cluster based ensemble classification for intrusion detection system, in: ACM International Conference Proceeding Series Part F128357, 2017, pp. 253–257, <https://doi.org/10.1145/3055635.3056595>.
- [48] A.A. Kurniawan, H.A. Santoso, M.A. Soeleman, A.Z. Fanani, Intrusion detection system as audit in IoT infrastructure using Ensemble learning and SMOTE method, in: Proceeding - 2019 5th International Conference on Science in Information Technology: Embracing Industry 4.0: Towards Innovation in Cyber Physical System 2019, ICSITech, 2019, pp. 205–210, <https://doi.org/10.1109/ICSITech46713.2019.8987524>.
- [49] A. Alhowaide, I. Alsmadi, J. Tang, Ensemble detection model for IoT IDS, Internet Things 16 (2021) 100435, <https://doi.org/10.1016/J.IOT.2021.100435>.
- [50] M. Eskandari, Z.H. Janjua, M. Vecchio, F. Antonelli, I.D.S. Passban, An intelligent anomaly-based intrusion detection system for IoT edge devices, IEEE Internet Things J. 7 (2020) 6882–6897, <https://doi.org/10.1109/JIOT.2020.2970501>.
- [51] T.K. Chien, L.Y. Chiou, S.S. Sheu, J.C. Lin, C.C. Lee, T.K. Ku, M.J. Tsai, C.I. Wu, Low-power MCU with embedded ReRAM buffers as sensor hub for IoT applications, IEEE J. Emerg. Sel. Top. Circuits Syst. 6 (2016) 247–257, <https://doi.org/10.1109/JETCAS.2016.2547778>.
- [52] H. Jayakumar, A. Raha, J.R. Stevens, V. Raghunathan, Energy-aware memory mapping for hybrid FRAM-SRAM MCUs in intermittently-powered IoT devices, ACM Trans. Embed. Comput. Syst. 16 (2017), <https://doi.org/10.1145/2983628>.
- [53] Why we recommend you do not use the Newman-Keuls multiple comparison test - FAQ 1093 - GraphPad, (n.d.), <https://www.graphpad.com/support/faq/why-we-recommend-you-do-not-use-the-newman-keuls-multiple-comparison-test/> (accessed January 25, 2024).
- [54] F.Z. Nakach, H. Zerouaoui, A. Idri, Hybrid deep boosting ensembles for histopathological breast cancer classification, Health Technol. (Berl) 12 (2022) 1043–1060, <https://doi.org/10.1007/S12553-022-00709-Z/TABLES/11>.
- [55] H. Zerouaoui, A. Idri, O. El Alaoui, A new approach for histological classification of breast cancer using deep hybrid heterogeneous ensemble, Data Technol. Appl. (2022) 1–34, <https://doi.org/10.1108/DTA-05-2022-0210>.
- [56] R.G. Mantovani, T. Horvath, R. Cerri, J. Vanschoren, A.C.P.L.F. De Carvalho, Hyper-parameter tuning of a decision tree induction algorithm, in: Proceedings - 2016 5th Brazilian Conference on Intelligent Systems 2016, BRACIS, 2017, pp. 37–42, <https://doi.org/10.1109/BRACIS.2016.018>.
- [57] A. McCallum, K.N.-A.-98 workshop on learning for, undefined, A comparison of event models for naive bayes text classification, Yangli-feasibility.ComA McCallum, K NigamAAAI-98 workshop on Learning for Text Categorization, 1998yangli-feasibility.Com (n.d.), <http://yangli-feasibility.com/home/classes/fd2022fall/media/aaaiws98.pdf>, 1998 accessed March 14, 2025.
- [58] G.H. John, P. Langley, Estimating continuous distributions in bayesian classifiers, (2013), <https://arxiv.org/abs/1302.4964v1> (accessed March 14, 2025).
- [59] Toubkal SuperComputer, (n.d.), <https://toubkal.um6p.ma/> (accessed July 25, 2024).
- [60] T. Whare W-ananga, W. Hamilton, M.A. Hall, Correlation-based feature selection for machine learning, <https://researchcommons.waikato.ac.nz/handle/10289/15043>, 1999 accessed February 16, 2023.
- [61] P. Fusco, G.P. Rimoli, M. Ficco, TinyIDS - an IoT intrusion detection system by Tiny Machine Learning, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 14816, LNCS, 2024, pp. 71–82, https://doi.org/10.1007/978-3-031-65223-3_5/FIGURES/3.



Ali Idri received his Ph.D. degree in cognitive computer sciences from the University of Quebec, Montreal, QC, Canada, in 2003. He is a Full Professor at the Computer Science and Systems Analysis School (ENSIAS, University Mohammed V, Rabat, Morocco). He is also an Affiliate Professor at the Faculty of Medical Sciences of Mohammed 6 Polytechnic University (FMS-UM6P). He has published more than 300 papers in several international journals and conferences. His research interests include software engineering, machine learning, cybersecurity, and medical informatics.



Abderahmane Hamdouchi is a PhD student at Mohammed VI Polytechnic University (UM6P-Morocco). He earned a BSc in mechanical engineering from the Royal Military Academy (ARM-Morocco). He received his master's degree in data sciences and decision support from Mohammed VI Polytechnic University (UM6P-Morocco). His primary research interests are machine learning in cybersecurity, ensemble learning, and TinyML.